WORKFLIOW SCHEDULING USING MEMETIC ALGORITHM

A
Project Report
Submitted in partial fulfilment of the
Requirements for the award of the Degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE & ENGINEERING

By

Deekshitha Pembarthi

1602-21-733-073

Varsha Varaganti

1602-21-733-129

Under the guidance of

Sriperambuduri Vinay Kumar

Assistant Professor

Department of Computer Science & Engineering

Vasavi College of Engineering (Autonomous)

(Affiliated to Osmania University)

Ibrahimbagh, Hyderabad-31

2025

Vasavi College of Engineering (Autonomous)

(Affiliated to Osmania University)

Hyderabad-500031

Department of Computer Science & Engineering

DECLARATION BY THE CANDIDATE

We, **Deekshitha Pembarthi** and **Varsha Varaganti** bearing hall ticket numbers,1602-21-733-073 and 1602-21-733-129 hereby declare that the project report entitled **"Workflow Scheduling using Memetic Algorithm"** under the guidance of **Sriperambuduri Vinay Kumar, Assistant Professor,** Department of Computer Science & Engineering, VCE, Hyderabad, is submitted in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering**.

This is a record of bonafide work carried out by us and the results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Deekshitha Pembarthi

1602-21-733-073

Varsha Varaganti

1602-21-733-129

Vasavi College of Engineering (Autonomous)

(Affiliated to Osmania University)

Hyderabad-500 031

Department of Computer Science & Engineering

BONAFIDE CERTIFICATE

This is to certify that the project entitled **"Workflow Scheduling using Memetic Algorithm"** being submitted by **Deekshitha Pembarthi** and **Varsha Varaganti,** bearing **1602-21-733-073** and **1602-21-733-129,** in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science & Engineering is a record of bonafide work carried out by them under my guidance.

S. Vinay Kumar	**Dr. T. Adilakshmi Professor**
Assistant Professor	**Professor & HOD**
Dept. of CSE	**Dept. of CSE**

External Examiner

ACKNOWLEDGEMENT

An endeavour of a long period can be successful only with the advice of our parents and well-wishers. We now took this opportunity to express our deep gratitude and appreciation to all those who encouraged us for the successful completion of this project.

We wish to express our sincere gratitude to **Dr. S. V. RAMANA M.Tech., Ph.D., Principal**. His consistent help and encouragement to complete the project work.

Our special thanks to **Dr. T. ADILAKSHMI M.Tech., Ph.D., Head of the Computer Engineering Department, Vasavi College of Engineering** for her valuable guidance and suggestions in analysing and testing throughout the period, till the end of project work completion.

We are thankful to our guide **S. Vinay Kumar, Assistant Professor, in the Department of Computer Engineering, Vasavi College of Engineering** for his valuable guidance and suggestions in analysing and testing throughout the period, till the end of project work completion.

Last but not least, we would love to thank my parents and friends who helped us, directly or indirectly, in finalizing this project within the limited time frame.

Abstract

Cloud computing has transformed modern data processing by offering scalable, agile, and economical access to distributed computing resources. Workflow scheduling, one of the most critical issues in this domain, is assigning a set of interdependent tasks to virtual machines (VMs) in such a manner that the execution time, cost, and resource usage are optimized. Workflow scheduling being an NP-hard issue is computationally expensive and complex, especially in heterogeneous and dynamic cloud environments. Traditional metaheuristic strategies such as Genetic Algorithms (GA) and Particle Swarm Optimization (PSO) have been widely used for this issue. However, these methods are prone to defects such as premature convergence and becoming trapped in local optima, which reduces their efficiency in finding high-quality solutions.

To overcome these limitations, this project proposes a workflow scheduling approach based on a Memetic Algorithm (MA), a metaheuristic that inherits the global exploration capability of Genetic Algorithms and the local exploitation capability of Hill Climbing. The Memetic Algorithm optimizes the search process by employing adaptive local search techniques that improve candidate solutions, enabling it to adaptively respond to workload variations and improve decision-making in task scheduling. The proposed approach aims to decrease makespan, reduce energy consumption, and lower execution costs while maximizing resource utilization and service-level agreements (SLAs). This project is simulated using CloudSim, a widely used cloud simulation platform, to evaluate the performance of the Memetic Algorithm against the traditional Genetic Algorithm. Results indicated that MA improved response time, scheduling accuracy, and system efficiency in a significant way. Despite the inherent complexities of computation time and scalability in large-scale systems, the results validate the effectiveness and reliability of Memetic Algorithms in addressing workflow scheduling problems in cloud computing. This paper emphasizes the potential of hybrid methods in designing smart, adaptive, and cost-efficient scheduling mechanisms for future cloud-based systems.

TABLE OF CONTENTS

1. Introduction.....1
    1.1. Background.....2
    1.2. Motivation…….…….....3
    1.3. Objectives.....4
    1.4. Scope……5
    1.5. Methodology……6

2. Literature Review……..……...….9

3. Proposed methodology…….....13

4. Experiment Results ……..……37

5. Conclusion and Future Work ……..…….....43

References…….....47

1. INTRODUCTION

Cloud computing has advanced at a startling pace and transformed how resources are allocated to their data centers and applications are deployed. Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS) are some of the service models offered by cloud platforms. The abstraction of physical hardware offers flexible and scalable services. These models assist users in managing and dynamically launching applications without imposing significant costs in IT infrastructure at the outset.

One of the foremost challenges pertaining to this issue is workflow scheduling, which involves the assignment of dependent tasks to appropriate cloud resources in an efficient and scalable manner. Overall execution time (makespan), operational expenditure energy efficiency, and compliance with QoS constraints are usually the aims considered while working on this problem.

Due to the high level of ever present complexity and fluidity of cloud environments, traditional scheduling algorithms tend to face challenges in fully utilizing cloud resources. Most based on adaptive iterated greedy designs fail to seek and exploit successful areas within a region of the solution space, neglecting possibilities for better results. These algorithms also tend to trap within their distinctive characteristic cloud systems.

This scheduling strategy uses a Memetic Algorithm, which combines the global search efficiency of Genetic Algorithms with more localized techniques such as Hill Climbing. This hybrid approach optimally balances exploitation and exploration, accelerates the search process. It also improves the precision and accuracy of the derived solutions. The proposed method is efficient in cost and system responsiveness, demonstrating better response times in complex workflow scheduling problems.

Work-flow Scheduling in Cloud Computing is the process of assigning interdependent tasks with constraints like cost, make-span, energy usage & resources availability. The objective is to maximize system throughput, satisfy deadlines, and ensure cost-effectiveness. Classic scheduling algorithms like First Come First Serve , RR & even

more sophisticated metaheuristics like GA & Particle Swarm Optimization have been used for this problem.

To address these issues the project propose a hybrid metaheuristic strategy through a MA a evolutionary algorithm that synergizes the global search capability of GA with the fine-tune capacity of local search algorithms such as Hill Climbing . The synergy allow Memetic Algorithm to traverse huge solution of spaces with out suffering from pre-mature convergence a known problem with Genetic Algorithm & PSO based scheduler.

1.1 BACKGROUND

The emergence of Cloud Computing have ushered in unmatched capabilities in the areas of distributed processing, scalable storage, and cost-effectiveness. Cloud service providers such as AWS (Amazon Web Services) & Microsoft Azure has transformed the way organizations implement large-scale applications like data analytic & machine learning. At the core of this transformation is the concept of Work-flow Scheduling which govern how computational task many of which depend on each other are allocated to the under-lying resource.

During initial cloud adoption periods, static scheduling techniques like (First Come First Serve) FCFS & Round Robin (RR) were adequate as a result of deterministic workloads and comparatively small-scale environments. With the increase in demand and heterogeneous and dynamic workloads, these methods were found inadequate. It was switched to heuristic and metaheuristic techniques like Genetic Algorithm(GA), PSO &ACO by researchers and practitioners because they could give almost optimal solutions in complicated search spaces.

Even though they had benefits, the algorithms showed significant limitations. GAs, though good at global solution exploration, converged prematurely and lost sight of better solutions because they lacked refinement. PSO converged more quickly but had difficulty with maintaining diversity during search. These factors prompted the necessity for hybrid approaches that integrate the capacity for exploration (Glocal) with exploitation(Local).

Come Memetic Algorithms (MAs)—a more powerful variant of evolutionary computation that integrates local search methods into global search paradigms. MAs not only maintain diversity also improve individual solutions using methods such as Hill Climbing. This leads to faster convergence, better quality schedules, and dynamic environment robustness. In cloud computing, MAs facilitate adaptive scheduling that effectively adapts to changing work-load intensity, Vitual Machine availability & Quality of Service requirement.

Significance of workflow scheduling optimization cannot be overemphasized. Poor scheduling results in higher power usage, SLA breaches, and increased expenses. With growing cloud adoption, particularly in industries like Scientific- research & Healthcare the demand for advanced schedulers such as MAs is on the rise.This project takes this a step further, suggesting a MA(Memetic Algorithm) for Work-flow Scheduling improves on the limitations of existing methods and keeps pace with the changing nature of cloud computing.

1.2 MOTIVATION

Increased use of cloud computing to achieve (HPC)High-Performance Computing , analytics & hosting applications in real time has subjected great pressure towards optimal utilization of cloud resources. Workflow scheduling ranks among the quintessential challenges for this field whereby the process involves matching a series of dependent operations with available processing resources taking into account constraints ranging from cost and time to power usage and reliability of the systems.

Legacy scheduling schemes such as Particle Swarm Optimization & Genetic Algorithm provide hopeful heuristic approaches to solving workflow scheduling. These algorithms, however, suffer from certain drawbacks—most notably premature convergence, local optima trap, and absence of fine-tune mechanism in dynamic setting. These issues can considerably impede the performances of cloud based application particularly when workflow are extensive in scale & have intricate dependencies.

Our inspiration arises from the requirement to design a smart scheduling mechanism that not only addresses the limitations of current methodologies but also adapts to shifting cloud states in a dynamic manner. Our project exploits a Memetic Algorithm a metaheuristic algorithmic method that inherits the global exploration properties of GA while leveraging the local fine-tuning advantages of Hill Climbing. MAs has prove to be exceptional in other optimization applications, and their potential for Cloud Work-flow Scheduling is interesting but untapped. By using an MA based schedule we expect performance in terms of lower makespan, decreased execution cost, better load balancing, and enhanced scalability.

In addition with increasing energy efficiency awareness and green computing, minimizing power consumption through optimal scheduling is a key driving factor. Optimized task allocation reduces idle time for VMs and increases server usage, thereby leading to greener methods of computing. It is motivated by vision of maximizing cost savings, flexibility, and reliability in workflow execution within cloud computing frameworks, with applicability to both research within academia and industrial application.

1.3 OBJECTIVES

The main goal of the project is to design and test intelligence of Workflow Scheduling algorithm based on Memetic Algorithm (MA) for effective task managements in the Cloud Computing. This main goal is divided into the following specific goals.Design & Implementation of a Hybrid Scheduler: Integrate Genetic Algorithm with Hill Climbing to create a hybrid optimized model that avoids local optima problems.Optimization of Make-span & Execution Cost – It will minimize the overall make-span & cost of using cloud resources by scheduling tasks more smartly.

Improved Resource Utilizations - Schedule work-load between virtual machine optimally to avoid over-loading & underutilization thus maximizing load-balancing. Improved Scheduling Accuracy - Refine schedule outcome through a local- search method to achieve more accuracy & high-quality tasks to resource mapping. Scalability - Design a scheduling mechanism that is perform an even for large-scale workflows

involving 1000s of task. Application for Simulation for Evaluation – Utilization of Cloud-Sim as the simulation engine for evaluating & comparing the performances of the Memetic Algorithm with conventional algorithms such as Genetic Algorithm & PSO.

Support-Fault Tolerance - Provide support for tasks to dynamically manage failures and unavailability of Virtual Machine ensuring workflow execution is consistently. .Energy Efficiency – To minimize un-necessary computation and idle VM time to conserve energy and render the scheduling process more eco-friendly. User-Centric Design - Develop system that are easy to use and flexible & facilitating seam-less integration into current cloud platforms for researcher & service provider.

1.4 SCOPE

This project focuses on solving the workflow scheduling problem in cloud computing environments with a Memetic Algorithm (MA)-based solution. The scope here involves the design, development, and testing of a hybrid metaheuristic algorithm that involves both the global search achieved from Genetic Algorithms and the local refinement potential of Hill Climbing to optimize task scheduling across distributed cloud resources. The research is particularly concentrated on multi-objective optimization technique, with the main aim of decreasing important performance indicators like execution time, cost, and energy consumption, as well as improving client satisfaction and requirement with Service Level Agreements (SLAs). The algorithm runs under a realistic cloud computing condition, such as resource heterogeneity, network latency, and dynamic workload overheads, to ensure real-world applicability in the cloud environments.

The scope of the project involves:

- Design of a workflow scheduling algorithm on the basis of the Memetic Algorithm.
- Testing and simulation of the algorithm on CloudSim, a cloud computing simulation framework.

- Performance comparative analysis with conventional scheduling methods such as Genetic Algorithms.
- Analysis of the major parameters such as makespan, cost, energy efficiency, and resource utilization.

1.5 METHODOLOGY

The Memetic Algorithm (MA) is integrated with the global search traits of traditional Genetic Algorithms (GA) and finely tuned local optimization strategies like Hill Climbing. Usually, the integrated process allows the algorithm to look into wide search spaces and optimize different facets of solutions in a fine-tuned manner. MA improves the workflow scheduling problem in cloud computing, which is NP-hard because it requires a large meticulous assignment of tasks and resources in order to minimize cost,makespan, resource utilization, while honouring strict deadlines and SLAs in a significant way. The functioning of the MA is a process of a series of specified steps, each of them contributing to improve the ultimate scheduling outcome.

i. Initialization

The algorithm initiates with a random initial population of potential solutions, each being a specific type of the workflow tasks onto available cloud resources (e.g., virtual machines). The solutions are depicted such that each workflow task is connected to a VM, and consequently, the entire execution plan is constructed. Diversity of the population is very important because it enables to cover a large region of possible solutions bythe algorithm. By covering extra regions of the solution space early, the algorithm minimizes the chances of premature convergence and maximizes the chances of finding a globally optimal solution

ii. Fitness Evaluation

Each population solution is evaluated by a fitness function that tries to capture the quality of scheduling. Many significant metrics are taken into account by the fitness function:

Execution Cost: Total cost incurred while executing the tasks on cloud resources.

Makespan: Cumulative time required to finish the entire workflow.

Resource Utilization: The extent to which available resources are being utilized effectively.

The fitness function also encompasses real-world constraints, i.e., task dependencies, deadlines, and SLAs. The constraints ensure that only realistically feasible and implementable schedules are being taken into account, and hence the resultant solutions are made more realistic and applicable.

iii. Selection

For constructing the next generation, a selection is performed. This one selects parent solutions on the basis of their fitness values. The roulette wheel method of selection is widely employed wherein the chance for a solution n to be selected is based on its fitness score. It fulfils the requirement to:

- Use strong solutions by providing them with a greater opportunity for reproduction.
- Maintain the population diverse to prevent stagnation as well as the local optima traps.
- Preserving diversity is especially important in an effort to keep the search space dynamic and avoid the algorithm converging too quickly to locally optimal solutions.

iv. Crossover and Mutation

After parent selection, the genetic operations—crossover and mutation—are applied to produce new offsprings in terms of schedules that bring some variation and force exploration of other scheduling patterns

Crossover: This operation consists of sampling pieces of two or more parent solutions and mixing them to generate new individuals. Through the mixing of the attributes of good-performing solutions, crossover encourages the creation of new and potentially

improved scheduling setups. Various crossover strategies, including single-point or multi-point crossover, may be employed based on the encoding scheme.

Mutation: Mutation adds a random, small-scale alterations to a solution, i.e., assigning an activity to a different available VM. This operation is very important in avoiding the algorithm getting stuck in an area of the space that has a lot of suboptimal solutions. It helps in boosting the diversity of the population and enables the algorithm to search new areas of the search space, hence improving the speed and the scope of the search process.

v. Local Search with Hill Climbing

The thing that distinguishes the Memetic Algorithm from the basic Genetic Algorithm is the addition of a local search technique—a Hill Climbing, in our case—as a secondary level of post-processing procedure for each child generated in each iteration. Once the genetic iterations have generated the child solution, it is optimized by local search.

The Hill Climbing technique searches for nearby solutions step by step by making small, incremental changes (e.g., exchanging the resource allocation of a data centre of a task). If there are any changes in an improvement according to the fitness function, it is accepted and executed; otherwise, it is rejected. The local search is continued until no further improvement can be achieved in the local neighbourhood.

The greatest strength of this strategy is that not only does it make all the offspring distinct but also locally optimal, which improves the quality of the population overall over a period of time. By combining global exploration (using GA) and local exploitation (using Hill Climbing), the MA finds an ideal balance that accelerates convergence towards high-quality solutions without losing solution diversity.

2. LITERATURE REVIEW

Cloud computing consists of an base model with multiple data centers where numerous virtual machines are hosted. Each VM has different amounts of RAM, CPUs, and storage. The cloud works on a pay-as-you-go basis, so resource usage and high operational costs are optimised. Task scheduling that is not optimally designed can result in routing all tasks to all available VMs, and therefore raise operational costs. Many scheduling algorithms that have been created to improve resource allocation in cloud environments, including but not limited to Particle Swarm Optimization, Genetic Algorithms, and the Crow Scheduler, all aimed at minimizing execution time and the costs associated with the scheduling workflows in cloud computing.

The advance & efficient type of GA(Genetic Algorithm) has been developed for the scheduling of workflow in the IOT to be done under fog cloud environment with hard deadline In this approach called IOT based algorithm a rescheduling heuristic apply on associated with schedule searching & repairing the dependency checking, communication delay , critical path executions & performance evaluations. The method is demonstrates with in detailed observed simulation for performing better than previous algorithm in cost of execution, resource cost & in meeting strict deadline.

The RCSECH and RSECH are the two algorithms that have been developed with the main purpose of assisting workflows scheduling in multi layered computing environments i.e. mist, edge, fog, and cloud systems. These techniques focus on controlling real time workflows through the optimal combination of overlapping goals. The results include improved reliability, lowered energy usage, reduced execution costs and increased data security while still meeting strict deadlines. RCSECH is remarkable because of its addition of reliability enforcement and task duplication that makes the system more reliable and scalable. Experimental and practical results prove meaningful improvements like increased reliability of 7%, decreased energy of 8%, and more than 15% reduction in costs. These readings validate the effectiveness of the obtained solutions.

The Hybrid Metaheuristic Multi Objective Workflow Scheduling Algorithm is meant for optimizing task scheduling in a cloud fog collaborative environment with a focus on IoT applications that requires low latency. It attempts to solve the three conflicting objectives of reducing energy usage, execution costs, and makespan or execution time by leveraging the best features of Harmony Search and Genetic Algorithms. HMWSA uses topological sorting for preserving the order in task execution, promising that the dependent tasks are executed in the correct order. Based on Montage's performance, simulation experiments show that this algorithm is more efficient at balancing the defined objectives when compared to standard algorithms, especially in heterogeneous resource computing environments.

A new scheduling technique that uses cloud resources in a complementary way has been developed to merge with the fog environments with limited resources. This method focuses on the management of real time computation to improve intense IoT workflows. A heuristic approach to resource allocation is proposed that will be balancing the processing requirements and data transfer reduction simultaneously. By adjusting the impact of these elements in the process, the scheduling achieves favorable conditions with varying system dynamics and workloads while operating. Comparative analysis shows that this approach is superior over all fog only solutions, especially when dealing with high data volumes and strict timelines.

The Memetic Algorithm is an addition of traditional Genetic Algorithms, with the distinguishing feature of employing a local search strategy, a hill climbing algorithm, to improve solutions during global optimization in a potential manner. Such hybrid methods are developed for complicated workflow scheduling problems in the cloud computing environment to reduce makespan, execution costs, and work within tight deadlines. By decreasing global exploration to employ localized exploitation balancing, MA addresses the dependencies of complex large scale scientific workflows in a more effective way. A comparative results of continuous algorithms versus MA algorithms shows that using the Cloud Sim simulation environment, MA performs better when compared to other in evaluating execution accuracy, efficiency, and solution accuracy in relation to standard Genetic Algorithms and Particle Swarm Optimization.

The Discrete Non Dominated Sorting Genetic Algorithm II (DNSGA II) is a relatively new approach that aims to enhance the efficiency of task allocation in fog cloud computing environments. In addition to this, unlike its predecessor, DNSGA-II will implement crossover and mutation operations in discrete rather than continuous format, increasing the precision and accuracy of task to resource mapping in distributed systems. This algorithm aims to achieve several goals, especially decreasing the execution time (makespan) and overall execution cost. Efficient resource distribution is obtained by allocating tasks to both fog and cloud layers utilizing DNSGA-II in a strategic manner. Experimental evaluations verify that this discrete version will perform well in the continuous NSGA-II and all other approaches in execution time, cost-efficient scheduling, and automation.

A hybrid task scheduling strategy that is established through the combination of both Particle Swarm Optimization (PSO) and Lion Algorithm (LA) is used to improve the performance in cloud computing environments. This combined algorithm aims to optimize task scheduling within the specified time slots and budget set by the users in a more scalable way. With the CloudSim simulation environment tool, the proposed algorithm has been evaluated with different workflow algorithms, depicting better performance in reducing execution time and enhancing resource utilization efficiently. This approach provides rapid convergence of PSO and effective search exploitation of LA that helps in overcoming the difficulties generated by real time cloud scheduling problems successfully.

The Hybrid Metaheuristic Multi objective Workflow Scheduling Algorithm (HMWSA) mostly deals with application workflows in cloud fog settings that require more sensitive responses in terms of latency. Through a combination of Harmony Search and Genetic Algorithms, HMWSA will collectively aim to optimize objectives which include reducing makespan, energy consumption, and execution costs in a significant manner. By using a topological sorting method, the algorithm also maintains task dependency, therefore improving overall scheduling process. The proposed algorithm will perform well in terms of the cost and latency. Through thorough simulations that utilize authentic workflows, it was noted that HMWSA performs all other methods in

cost, latency, energy usage, and resource allocation in an outstanding way, thus achieving efficient workflow scheduling.

Workflow-Scheduling in Cloud Computing focuses on efficient assigning task to resources within cloud tackle challenges like task to resource allocation, adherence to QoS requirement, reliable resource provisioning & managing system-failure. These complexity make work-flow scheduling an NP hard problem . By systematic analyze a range of existing algorithms & strategies the paper deliver crucial insight into addresses these difficulty. This make it a strong foundation reference for researcher aim to innovate & improve scheduling methods for distributed cloud environment.

This Algorithm was inspired by natural flashing pattern of fire flies used for communication & attraction & offers a promising meta-heuristic for workflow scheduling in cloud computing. By this natural selection behavior this algorithm seeks to minimize execution time & enhance over all resources efficiency. Simulation study also shows that the Fire-fly algorithm can significantly improves scheduling effectiveness & positioning it as competitive alternativeness to more conventional approach in cloud environment.

This study further present structured workflow scheduling algorithm in cloud computing , categorizing them based on the specific features & goals. It explores critical challenge that are un-predictable resources availability , diversity in task type & differ QoS expectations. By evaluating the advantage & draw-back of current method the paper highlight existing research gap & introduces for future investigations. This review serve a valuable guide for both researcher & industry professional aims to deep their understanding & development of workflow scheduling in cloud environment.

3. PROPOSED METHODOLOGY

Cloud Computing have transformed the IT(Information Technology) industry by provides demand access to a common pool of adjustable computing resources such as storage, processing capacity & network capacities. The resources are provided with virtualization technology so that customers can dynamically provision & scale service as per the requirements. The very nature of flexibility ,reliability & scalability of cloud computing render it suitable for any number of application ranging from data analytics & artificial intelligence to enterprise resource planning and real time web service.

The actual efficiency and cost-effectiveness of cloud environments, however, largely depend on the quality of scheduling of the tasks. Specifically, workflow scheduling is critical in ensuring that tasks that are interdependent are assigned to proper cloud resources such that the total cost is minimized, execution time (make-span) reduced & resource utilization optimize. A workflow would comprise several tasks each of which has dependency & different computation requirement. In-effective scheduling can result in resource underutilization, longer processing time, increased operation costs & lowered quality of service(QoS).

Classic scheduling algorithms such as (HC)Hill Climbing & (GA) Genetic Algorithm has found extensive application in optimization problems that includes cloud scheduling. Genetic Algorithm(GA) simulate natural evolution in which the population of candidate solution evolve from one generation to another by performing operations such as mutation, crossover, and selection. Genetic Algorithms(GA) are well suited for searching large search spaces and escaping premature convergence to local solutions. Nevertheless one of the main drawback is their poor convergence rate, and this can be a severe disadvantage in applications where time is critical. Genetic Algorithms(GA) also occasionally fail to refine solutions near the optimum because they are stochastic in the nature.

Conversely, Hill Climbing (HC) local search is an easy & effective local-search algorithm that enhances a single solution by iteratively stepping to neighboring solutions with higher fitness values. Although Hill Climbing converges rapidly and

performs well for small or less complex problems, it tends to get stuck in local optima, particularly when the problem space is large and multi-modal, as is typical in work-flow scheduling. In the system proposed, Memetic Algorithm(MA) begin with the initial population of potential scheduling solution. These solutions are evolved by Genetic Algorithm's operator & each off-spring resulting from this is subjected to a local search through Hill Climbing(HC) to improve it's performance further before moving on to the next-generation. The fitness of every solution is computed with respect to multiple objectives such as cost, execution time(make-span) & resource utilization to achieve a complete optimization processes.

By combining the strength of both global(Genetic algorithm) & local search approaches(Hill Climbing) Memetic Algorithm (MA)based workflow scheduling provides an efficient & effective solution for cloud resource utilization optimization, minimizing operational costs, & improving the overall performance of the cloud-computing system.

The system proposed in this work tackles the intricate issue of workflow scheduling in cloud computing environments through the utilization of the strength of a Memetic Algorithm . The issue here is to schedule a sequence of interdependent tasks to virtualized cloud resources in an efficient manner while seeking to optimize major performance factors like make-span(Execution cost), relative cost & utilization of resources. In cloud workflow applications, where activities tend to be dependent and have different computational needs, scheduling is essential to ensuring Quality of Service requirements & minimizing operational expenses.

Unique optimization methods such as GA (Genetic Algorithm) has proven their prowess in searching large solution spaces & conducting global searches. None-the less they are prone to slow convergence and easily get stuck in local optima produces inferior solutions particularly in massive-scale and dynamic cloud computing. Alternatively HC (Hill Climbing) is a good local search method that excels in using the neighbourhood of a particular solution to search for a better solution. In spite HC is not capable of exploration as required to effectively explore vast and intricate solution spaces and is also prone to becoming stuck in the local optima.

The Memetic Algorithm(MA) provide a hybrid solution by combining the global search power of Genetic Algorithm(GA) with the Unique local optimization method ability of Hill Climbing(HC). This synergy enables Memetic Algorithm to not just explore the solution spaces extensively but also to exploit potentially good regions intensively which result in high quality solutions & quicker convergence. In cloud work-flow scheduling this is especially useful since it trades-off exploration(GA) & exploitation(HC) which are the both critical to handle the complexity of large scale workflow.

This system start with the creation of a heterogeneous initial population in which every individual solution is a possible to map of work-flow tasks to accessible cloud resource. The solutions are assessed based on an objective function that takes into account major factors like as (make-span)execution time, cost effectiveness & usage of computational resource. To develop this population the Memetic Algorithm use conventional Genetic Algorithm operator like selection, crossover & mutation that encourage genetic diversity & push the population towards improving solution from one generation to the next.

The distinguishing factor between Memetic Algorithm (MA) & conventional Genetic Algorithm is the use of local search method that is Hill Climbing Algorithm (HC) at the individual level. Once, the Genetic Algorithm operator generate offspring the local search technique that is Hill Climbing(HC) is used to each new solution in order to further optimize it before it is accepted into the next generation. This hybridization greatly improves the algorithm convergence rate while improve the quality & robustness of the solutions. The local-search method that is Hill Climbing (HC) perform well specifically in optimizing solution that are already close to optimal which may otherwise be neglected by the pure dynamic unquie global search method.

In addition this method subject the system to be effective & more responsive to dynamic cloud setting where resource availability & prioritize of task can alter instantaneously. The potential for quick adjustment in response to alterations while sustaining high quality scheduling makes Memetic Algorithm(MA) an attractive solution to real world cloud work-flow scheduling management.

Advantages are -

- Balance exploration(Genetic Algorithm) & exploitation(Hill Climbing) of the search space through a combination of global & local search strategies.
- Faster convergence than the conventional Genetic Algorithm to the employment of local search technique.
- Prevention of local optima traps that are usual problem in solely global & local approaches.
- Scalability & adaptability to large scale & dynamic cloud environments with intricate task dependency.
- Enhanced (QoS)Quality of Service compliance, particularly in term of meeting deadlines and being within budget limit.
- More efficient use of resources along with reduced idle time & increased through-put

Algorithms used:

1.Genetic Algorithm

A Genetic Algorithm (GA) can be defined as an optimization technique rooted in the process of natural selection as seen in biological evolution. It is a sophisticated search heuristic that attempts to derive solutions to intricate problems by simulating evolutionary activities such as survival of the fittest and reproduction accompanied by mutation. In particular, Genetic Algorithms tend to be helpful in resolving issues that are inefficient by other conventional means.

The functioning of a GA starts with the population being initialized with arbitrary potential solutions that are referred to as chromosomes. Each solution, also referred to as an individual or genome, is evaluated through a given exercise using a fitness function on a relevant problem to gauge how well it solves the problem. From the fitness scores of these individuals, the selection process selects a few of the better performers, and these individuals reproduce, thus becoming the parents of the next generation.

In the recombination process (crossover), a selected group of individuals is paired and new offspring are formed, ensuring that both characteristics of the parents are passed down to the offspring. In order to preserve sufficient diversity among the population in the presence of genetic drift, and to counteract premature convergence, a mutation step applies random alterations to a few of the offspring. The new generation is then formed by including all or some of the previously existing population along with the newly created offspring, and this is repeated until certain conditions to stop are fulfilled, for instance reaching a target number of generations, or attaining a specific desired fitness level.

Cons of Genetic Algorithm:

- Low convergence speed, mostly for larger sets of problems or more complicated challenges.
- High computational costs associated while considering multiple individuals across generations.
- May not always guarantee optimal or best solutions (not protected against local optima).
- Designed fitness function is varying

2.Hill Climbing Algorithm

The Hill Climbing Algorithm is a heuristic optimization method for discovering the best solution by refining a singular solution. It is analogous to climbing a hill because an ascent on a hill requires that one’s current position is of a greater altitude than the previous according to an evaluation or fitness function. The algorithm initiates with arbitrary solutions which are then altered in a minimal way. If the solution obtained is better than the previous one, the algorithm accepts it. This will be repeated till an optimal solution is reached.

Like any other method, the hill climbing algorithm has its pros and cons. Unlike other methods, the algorithm's unique behaviour is that it evaluates neighbouring solutions and picks the best. This makes it easy and faster to converge solely to lower local optima

within small or moderately sized problems. This makes the algorithm unsuitable for large problem spaces. As only nearby solutions are evaluated, it always becomes impossible to go anywhere and reach a local optimum.

Cons of Hill Climbing Algorithm:

- Usually misses the opportunity of a global solution as it is stuck on a local one.
- A step in hill climbing is irreversible.
- Too many local peaks and valleys along with complexity makes the search inefficient.
- Can not handle problems that possess multiple optimal solutions.

3.Memetic Algorithm

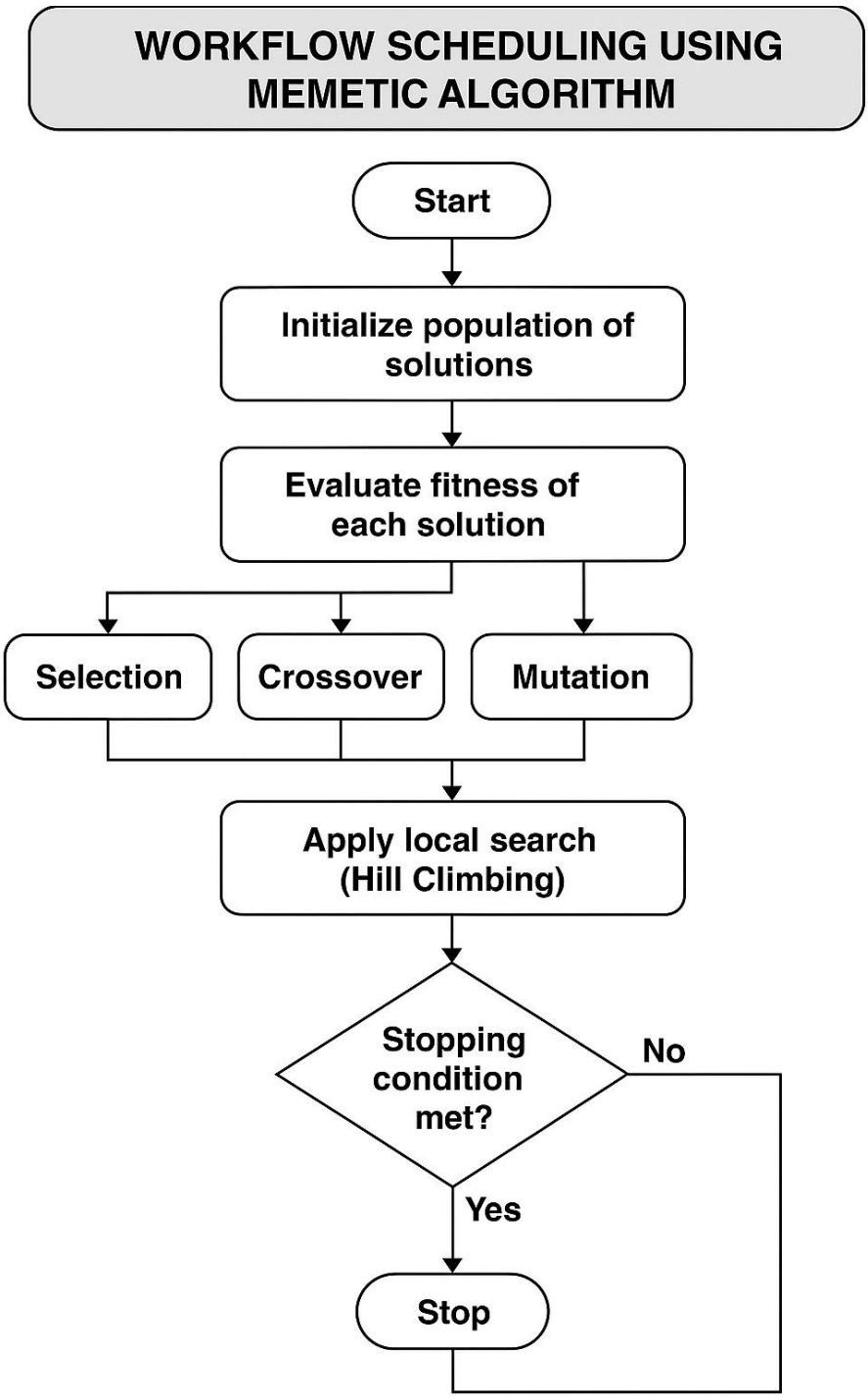
Memetic Algorithm (MA) is a sophisticated hybrid optimisation method which combines the global search power of Genetic Algorithm (GA) & the local search potency of methods such as Hill Climbing . Both are motivated by the evolutionary theory of natural selection and the principle of individual learning or improvement and, therefore, it is even more powerful and efficient than each method used alone.

The operation of a MA(Memetic Algorithm) begins in the same way as a GA(Genetic Algorithm). It starts with a random created population of individual where each individual is possible solution. The individual are assess with a fitness function for find out how well they are able to solve the problem. New offspring are created through selection, crossover & mutation simulating biological evolution.

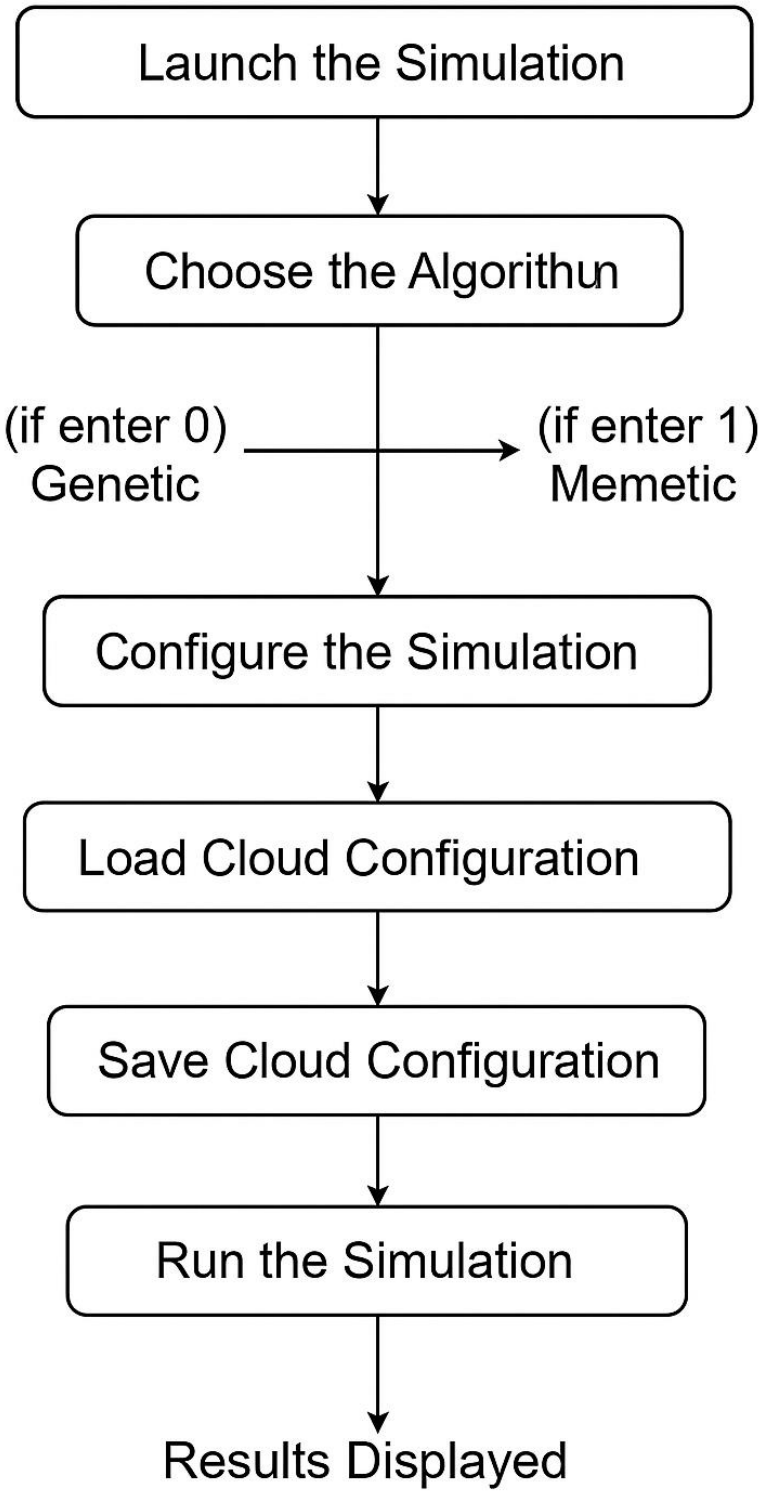
What distinguishes a Memetic Algorithm is the inclusion of local search. Once new individuals are generated a refinement step is performed based on local search methods like Hill Climbing(HC). This fine-tunes individual solutions, making them better prior to the initiation of the subsequent generation. Refined individuals then replace some portion of the existing population, and the process is repeated.

The primary strength of Memetic Algorithms is that they can integrate global exploration and local exploitation resulting in quicker and better-quality solutions. This

makes MAs especially suited to complex optimization problems where broad search and fine-tuning are both necessary.



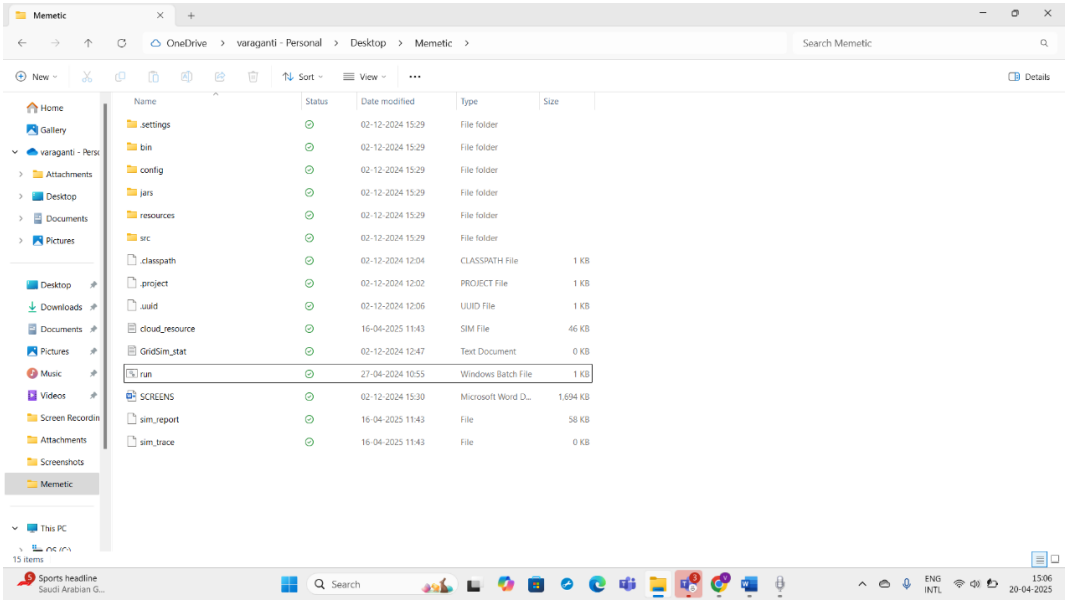
Implementation:



In conclusion, the Memetic Algorithm(Memetic) based workflow scheduling system presented in this paper provides a viable, efficient & adaptive solution for resource management in clouds. Through the combination of the benefits of global and local optimization method it provides a better performance than conventional scheduling techniques & is well adapted to deal with the complexities presented by contemporary cloud computing environment.

Steps to Execute Workflow Scheduling Simulation

i. Launch the Simulation



The initial step for the workflow scheduling process within the cloud simulation platform, is launching the simulation through the run.bat file. This execution is the primary and most significant action. This batch file acts as the simulation system's front-end entry and initiates all components required by the scheduling environment to operate smoothly.

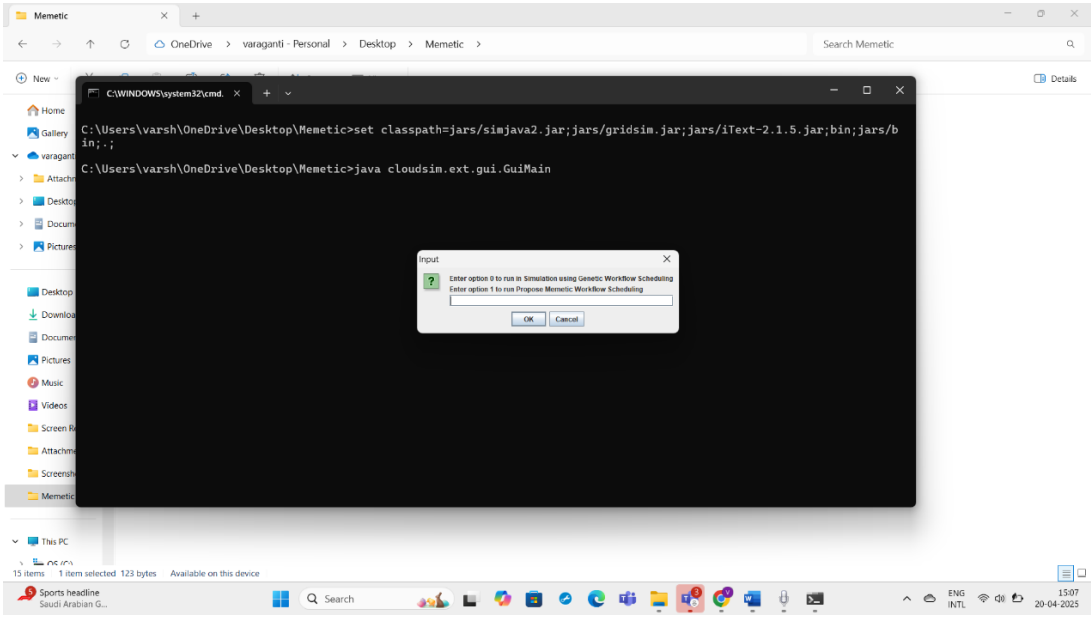
A .bat file(Batch) is the mostly used batch file(.bat) on Windows operating systems that holds a list of commands to be run by the command line interpreter manually. Within this project the run.bat file(Batch file) is specifically coded to reduce manual configuration automate the simulation component initialization & enhance

the user experience in a more straightforward way. Rather than having to compile Java files manually or type long commands into the terminal the user can simply double-click the run.bat file (.bat) to start the process.

The main reason for employing this file in the project scope within our use case is to add effectiveness, uniformity,integrity and ease of use. It will ensure that the simulation will started with the parameters, dependencies, and configurations required for initial simulation accuracy. It will set the classpath for the our required Java execution, load the necessary libraries, and bring up the GUI of the simulation while not allowing the user to access the command line. This is very relevant in complex environments like CloudSim which may have a mixture of various Java classes, additional JAR files, and configuration details.

run.bat allows for the automation of the startup processes by eliminating the user associated errors, guaranteeing the reproducibility, and saving time. This is particularly beneficial during the repeated testing and validating the scheduling algorithms. In other words, run.bat opens up the simulation part and functions as a starting point for the primary logic of the cloud workflow scheduling endeavour.

ii. Choose the Algorithm



Once the simulation is initiated after running the run.bat file a window will pop up (prompted) displaying a command for you to input the number corresponding to the algorithm you would like to use(0 or 1). This is an important choice moment in the simulation since it define the scheduling method (Genetic or Memetic) that will be used across the work-flow execution.

You are presented with two alternatives:

- Enter 0 :To execute the simulation based on the (GA)Genetic Algorithm.
- Enter 1 :To perform the simulation with the (MA)Memetic Algorithm

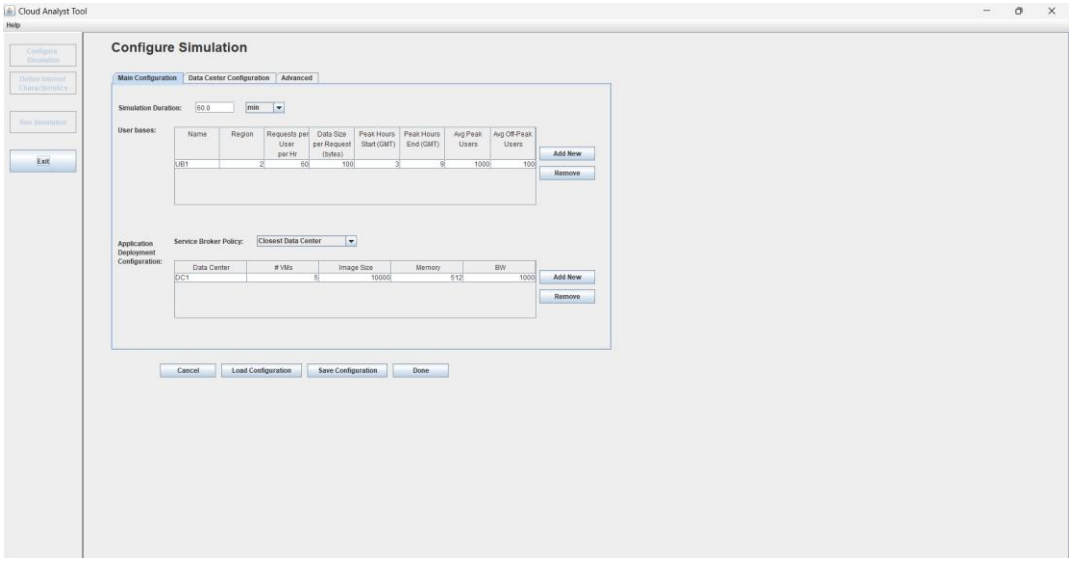
After entering the desired number (0 or 1) you would then need to click the "OKAY !!" button to accept your input & continue. This input is then passed into the program where the inner working are managed by a Java class called GuiMain (Main File which acts as a switch). The GuiMain class is a controller & switch facility which decode the input from the user. Depending on whether 0 or 1 was input it diverts the control flow to the respective algorithm(Genetic or Memetic). If 0 is selected the simulation continues the Genetic Algorithm(GA). If 1 is selected it goes to the Memetic Algorithm(MA).

The Genetic Algorithm(GA) is based on natural evolution & operates by evolving a population of possible solutions based on selection, crossover, & mutation. It is very good at searching large search space & preventing premature convergence but slower to converge to optimal solution.The Memetic Algorithm(MA) however is a blend of Genetic Algorithm (Global Search) & local search as Hill Climbing (HC). It not only searches globally like Genetic Algorithm but also refine the each solution locally to enhance convergence speed & accuracy.

For Genetic Algorithm:

iii. Configure the Simulation

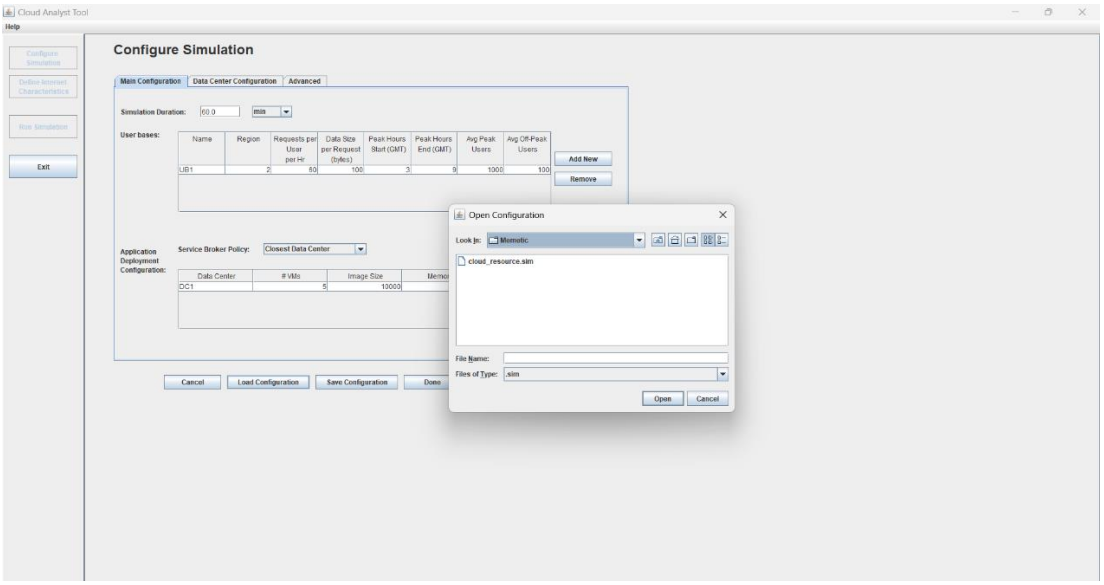
CloudAnalyst is a simulation tool that is designed to operate with CloudSim, by offering performance analysis for large-scale cloud applications in a more visual manner. In comparison to CloudSim, CloudAnalyst offers a GUI, that will enable the users to simulate cloud environments graphically, without using programmatical code. This is very useful for researchers, educators, and students focused on cloud computing.



In CloudAnalyst, users can set their configurations related to data centers, user bases, application workloads, and VM alThe interface of this tool includes buttons for setting up the simulation, specifying characteristics of the internet, such as latency and bandwidth, and executing the simulation. In the middle section of the tool screen, there is a graphical display of a world map, which has two pointers: UB1 (which is a user base located in Europe) and DC1 (which is a data center in North America). This configuration depicts the scenario of users situated in Europe interfacing with the cloud infrastructure located in North America. It is quite helpful while calculating response times and network latency.

CloudAnalyst is instrumental in assessing the important performance indicators which include the value of response time, the duration for processing data, and the total expenditure the final user would incur, in terms of relocating. The system also allows for the simulation of various load balancing policies to assess the cost-benefit of establishing data centers in different parts of the world.location policies. It will enable students and scholars to simulate real-world internet-based applications under various scenarios of cloud deployment. By far, the most outstanding feature of this tool is the graphical presentation of user bases and data centers using coordinates in world map.

iv. Load Cloud Configuration



Once the choice of the desired algorithm have been made in the above step the second most important step is set up the simulation environment. This is accomplish by click on the Configure Simulation button (in the simulation interface). When we click we will be asked to load a configuration file that specifies the complete infrastructure setup for the simulation(cloud resource sim file).

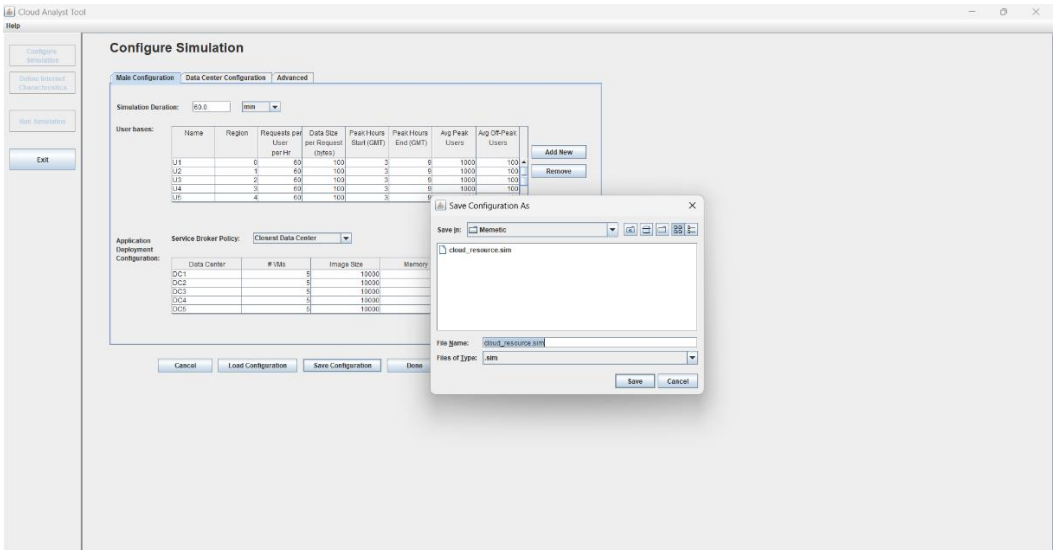
Click on the Load Configuration & browse to upload the file called cloud_resource.sim(cloud resource sim file). This file is a core part of the project it is the blueprint of the simulated cloud environment. It holds all the pre defined configurations that define the virtualized infrastructure, such as the number and types of data centres, (VM)virtual machines & (UB) user bases . These are vital because they emulate the actual cloud environments where the work-flow tasks would be scheduled & run.

With-out loading this configuration file the simulator won't have a context and an environment to give tasks. The cloud_resource.sim (cloudresouce.sim file) file guarantee the environment is regular, reproducible & that it reflects precisely the simulation parameters desired. Not only does it define the hardware resources it also defines users distribution as well as patterns of interaction of the users with the cloud infrastructure.

After loading you will notice that multiple user bases & data centres are now clearly configured in the simulation interface. This means that the virtual environment is now prepared to simulate the task scheduling and execution process.

In addition, loading the cloud_resource.sim file is a building block step since it sets up the cloud environment necessary for meaningful & correct simulation. It enables the scheduler to run within a specified infrastructure and facilitates the measurement of performance metrics such as cost, make-span (execution time) & resource utilization.

v. Save Cloud Configuration

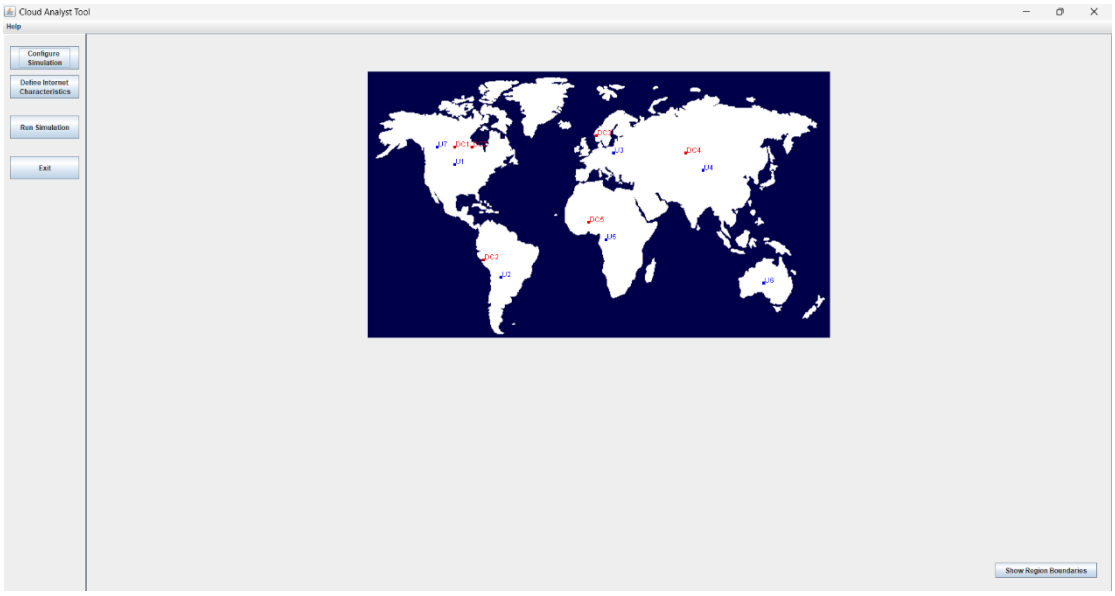


After the cloud_resource.sim file is successfully loaded into the simulation interface, the second significant step is to save the configuration. This is accomplished by merely clicking the "Save Configuration" button provided in the setup panel. While this may appear as a minor step, it is a vital step in maintaining the loaded environment settings and ensuring the simulator properly applies the infrastructure setup in the course of the workflow scheduling simulation.

Saving the configuration performs multiple task in the background. Firstly it commits the entire loaded structure includes data centers, VM(virtual machines) , user bases & their respective resource specification to the active memory of the simulation. This means that cloud simulation engine now recognize these component as part of the execution environments. Without saving even if the configuration file was loaded the simulator might not proceed correctly because it wouldnot register the infrastructure setup.

Secondly saving the configuration ensure that the cloud topology is fixed and that there are no accidental modifications and data loss. It assures that all the resources defined are available and ready to be utilized by the scheduling algorithm. Consistency across several simulation runs is also ensured through this step.

vi. View the Setup

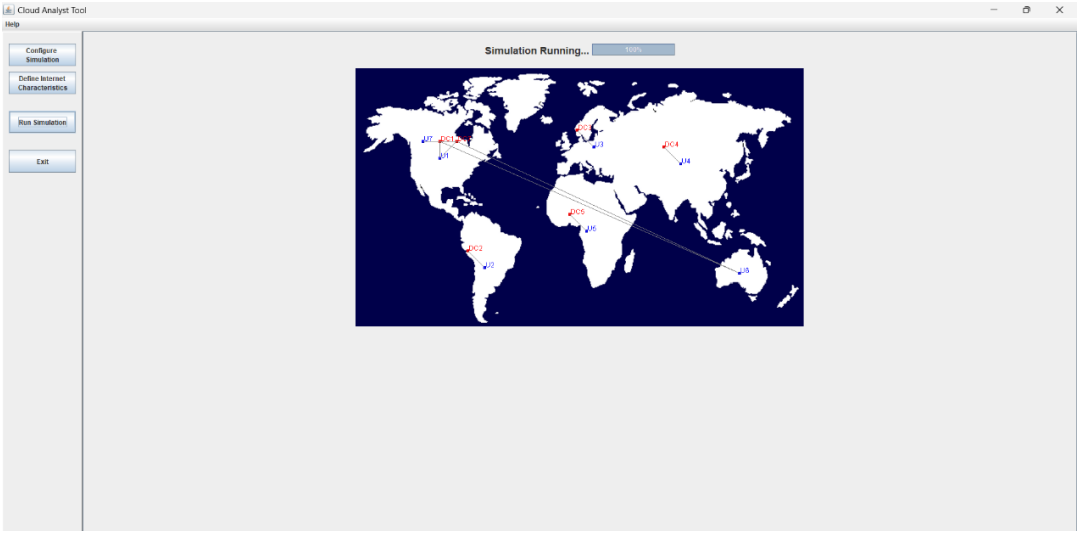


After the successful load & save operations of the configuration cloud_resource file the simulation environment is now in its complete initial state. It is at this point that you can visually see the configuration setup within the simulation interface. You will see displayed on the screen a graphical form of the cloud infrastructure, exhibiting several User Bases & Data Centres . These elements are typically depicted through color coded icons and figures so that it is simple to identify different elements within the cloud configuration.

The User Bases are various user groups or client systems that create workflow tasks. The tasks emulate the system load, for example, data processing, computation and service requests. The Data Centres, by contrast are where the Virtual Machines used to run these tasks reside. There may be different VMs in a single data centre with various specifications as regards CPU, memory & bandwidth.

This graphical display guarantees that the cloud infrastructure has been properly configured & that the necessary resources are available. The visualizing of the configuration at this stage is important since it acts as a confirmation of the final state prior to executing the simulation. It also facilitates confirmation that the configuration loaded accommodates the intended test conditions for workflow scheduling

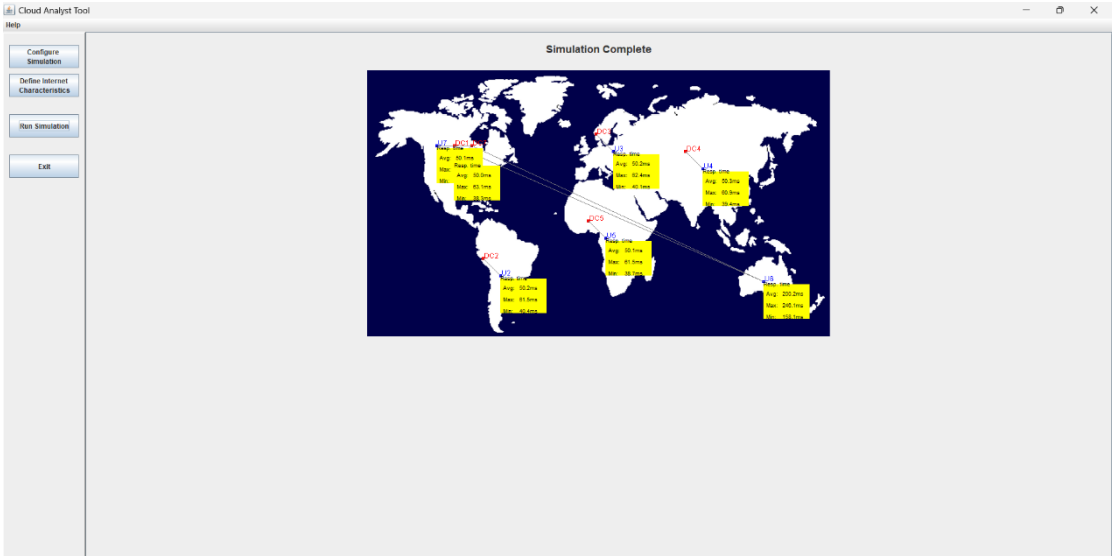
vii. Run the Simulation



After the configuration have been successfully saved ,loaded & the simulation environment is completely established with data centres and user bases, the next thing to do is to start the actual workflow execution by click on the Run Simulation button. This will initiate the core functionality of the system running the chosen scheduling algorithm in the configured cloud environment.

Once the workload simulation commences, (UB)User Bases begin to create and submit workflow tasks to the ready (DC) Data Centres using the established network connection. These tasks are real-world workloads of the cloud & are scheduled to run according to the previously selected algorithm either Genetic Algorithm & Memetic Algorithm. The simulator now proceeds to execute these tasks by assigning them to the best suited (VM)virtual machines in the available data centres.

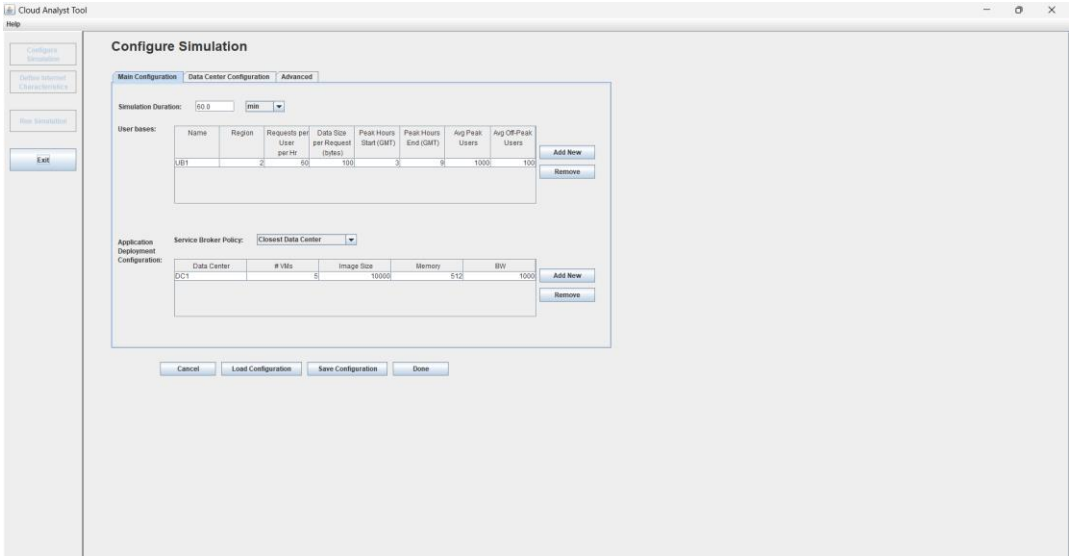
During simulating the system assesses various parameters like execution time (make-span) , response time, power usage & resource utilization. These value give you an idea of how effectively the algorithm is scheduling the tasks. You will also see real time graphics & statistics displaying how requests are being allocated and processed. This phase enables users to evaluate the performance and efficiency of the scheduling algorithm under simulated real world cloud computing environment.



For Memetic Algorithm:

viii. Configure the Simulation

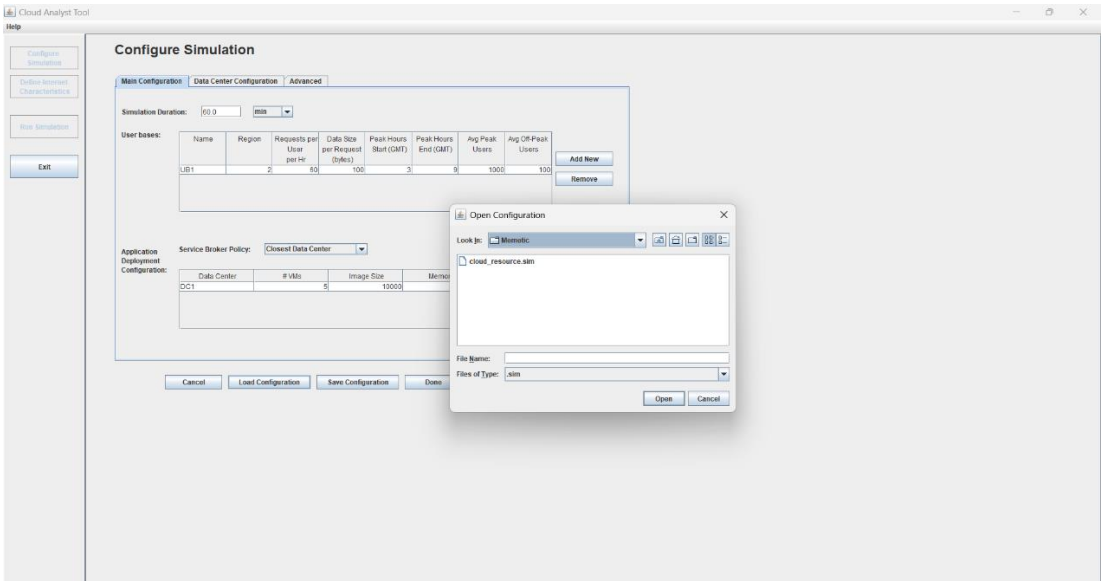
CloudAnalyst is a simulation tool that is designed to operate with CloudSim, by offering performance analysis for large-scale cloud applications in a more visual manner. In comparison to CloudSim, CloudAnalyst offers a GUI, that will enable the users to simulate cloud environments graphically, without using programmatical code. This is very useful for researchers, educators, and students focused on cloud computing.



In CloudAnalyst, users can set their configurations related to data centers, user bases, application workloads, and VM alThe interface of this tool includes buttons for setting up the simulation, specifying characteristics of the internet, such as latency and bandwidth, and executing the simulation. In the middle section of the tool screen, there is a graphical display of a world map, which has two pointers: UB1 (which is a user base located in Europe) and DC1 (which is a data center in North America). This configuration depicts the scenario of users situated in Europe interfacing with the cloud infrastructure located in North America. It is quite helpful while calculating response times and network latency.

CloudAnalyst is instrumental in assessing the important performance indicators which include the value of response time, the duration for processing data, and the total expenditure the final user would incur, in terms of relocating. The system also allows for the simulation of various load balancing policies to assess the cost-benefit of establishing data centers in different parts of the world.location policies. It will enable students and scholars to simulate real-world internet-based applications under various scenarios of cloud deployment. By far, the most outstanding feature of this tool is the graphical presentation of user bases and data centers using coordinates in world map.

ix. Load Cloud Configuration



Once the choice of the desired algorithm have been made in the above step the second most important step is set up the simulation environment. This is accomplish by click on the Configure Simulation button (in the simulation interface). When we click we will be asked to load a configuration file that specifies the complete infrastructure setup for the simulation(cloud resource sim file).

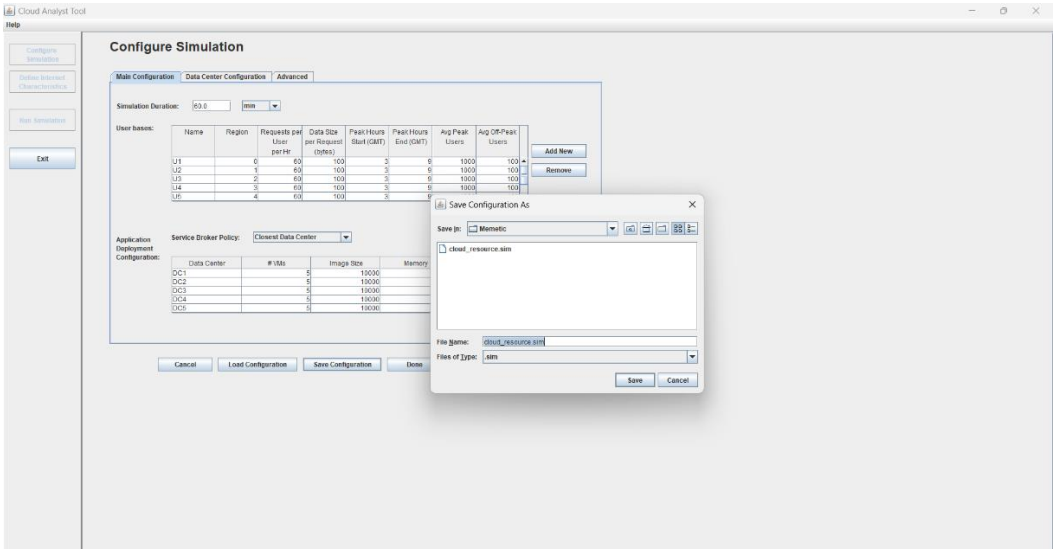
Click on the Load Configuration & browse to upload the file called cloud_resource.sim(cloud resource sim file). This file is a core part of the project it is the blueprint of the simulated cloud environment. It holds all the pre defined configurations that define the virtualized infrastructure, such as the number and types of data centres, (VM)virtual machines & (UB) user bases . These are vital because they emulate the actual cloud environments where the work-flow tasks would be scheduled & run.

With-out loading this configuration file the simulator won't have a context and an environment to give tasks. The cloud_resource.sim (cloudresouce.sim file) file guarantee the environment is regular, reproducible & that it reflects precisely the simulation parameters desired. Not only does it define the hardware resources it also defines users distribution as well as patterns of interaction of the users with the cloud infrastructure.

After loading you will notice that multiple user bases & data centres are now clearly configured in the simulation interface. This means that the virtual environment is now prepared to simulate the task scheduling and execution process.

In addition, loading the cloud_resource.sim file is a building block step since it sets up the cloud environment necessary for meaningful & correct simulation. It enables the scheduler to run within a specified infrastructure and facilitates the measurement of performance metrics such as cost, make-span (execution time) & resource utilization.

x. Save Cloud Configuration



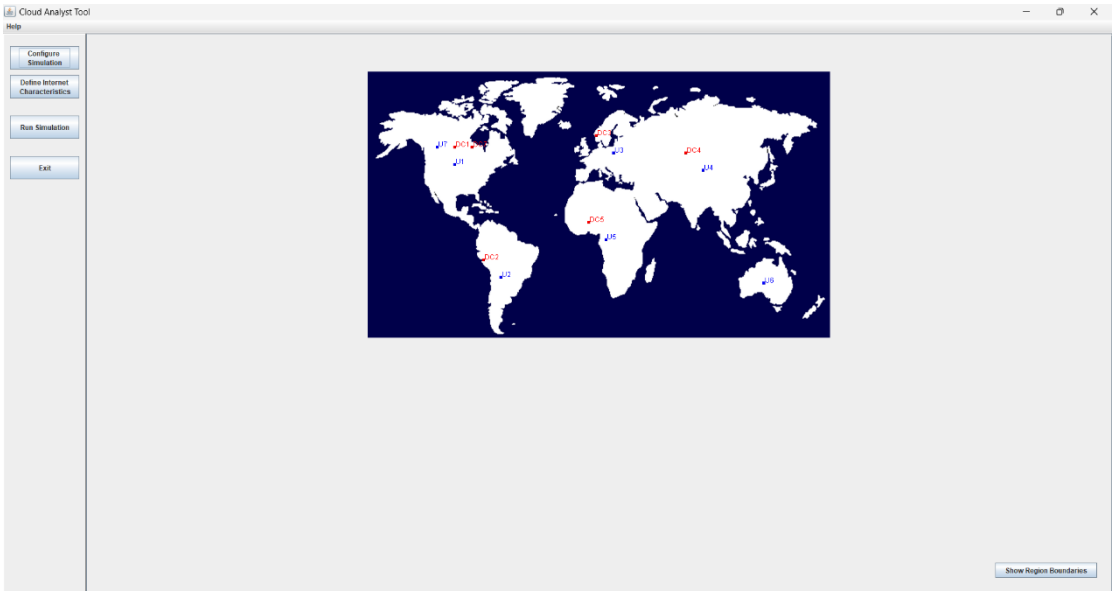
After the cloud_resource.sim file is successfully loaded into the simulation interface, the second significant step is to save the configuration. This is accomplished by merely clicking the "Save Configuration" button provided in the setup panel. While this may appear as a minor step, it is a vital step in maintaining the loaded environment settings and ensuring the simulator properly applies the infrastructure setup in the course of the workflow scheduling simulation.

Saving the configuration performs multiple task in the background. Firstly it commits the entire loaded structure includes data centers, VM(virtual machines) , user bases & their respective resource specification to the active memory of the simulation. This means that cloud simulation engine now recognize these

component as part of the execution environments. Without saving even if the configuration file was loaded the simulator might not proceed correctly because it wouldnot register the infrastructure setup.

Secondly saving the configuration ensure that the cloud topology is fixed and that there are no accidental modifications and data loss. It assures that all the resources defined are available and ready to be utilized by the scheduling algorithm. Consistency across several simulation runs is also ensured through this step.

xi. View the Setup



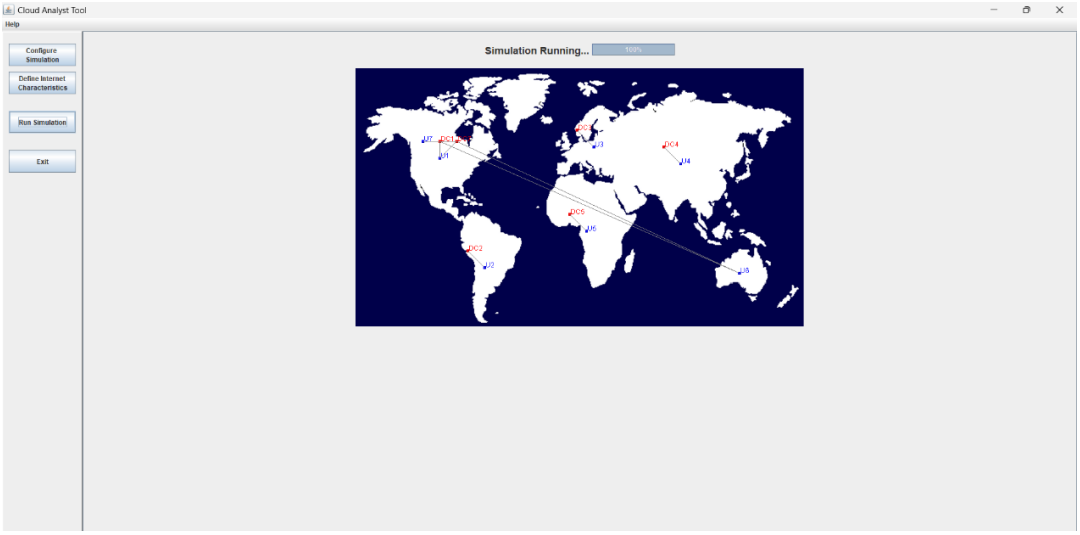
After the successful load & save operations of the configuration cloud_resource file the simulation environment is now in its complete initial state. It is at this point that you can visually see the configuration setup within the simulation interface. You will see displayed on the screen a graphical form of the cloud infrastructure, exhibiting several User Bases & Data Centres . These elements are typically depicted through color coded icons and figures so that it is simple to identify different elements within the cloud configuration.

The User Bases are various user groups or client systems that create workflow tasks. The tasks emulate the system load, for example, data processing, computation and service requests. The Data Centres, by contrast are where the Virtual Machines used

to run these tasks reside. There may be different VMs in a single data centre with various specifications as regards CPU, memory & bandwidth.

This graphical display guarantees that the cloud infrastructure has been properly configured & that the necessary resources are available. The visualizing of the configuration at this stage is important since it acts as a confirmation of the final state prior to executing the simulation. It also facilitates confirmation that the configuration loaded accommodates the intended test conditions for workflow scheduling

xii. Run the Simulation

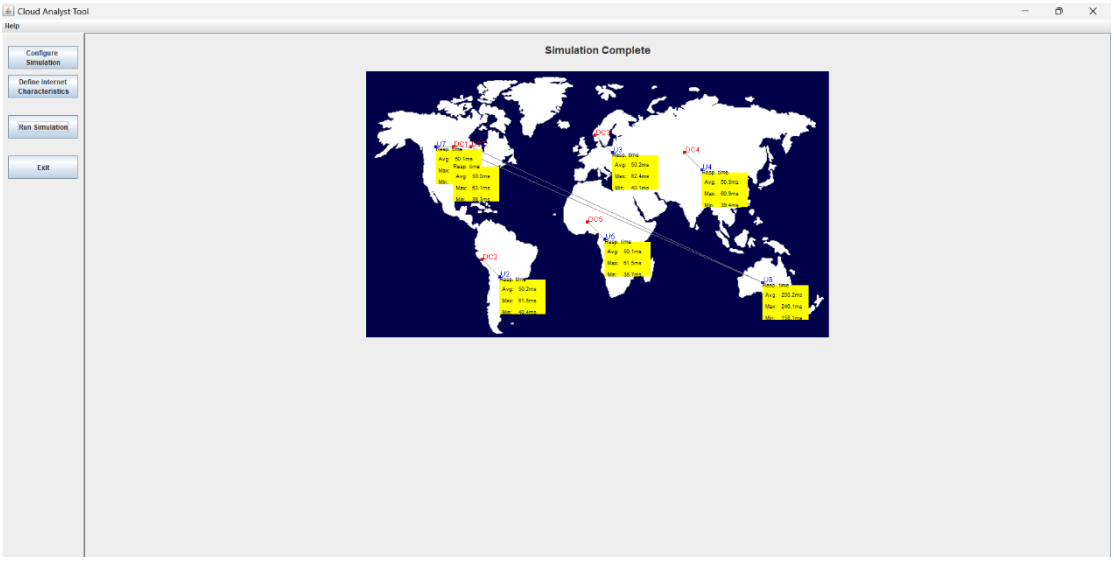


After the configuration have been successfully saved ,loaded & the simulation environment is completely established with data centres and user bases, the next thing to do is to start the actual workflow execution by click on the Run Simulation button. This will initiate the core functionality of the system running the chosen scheduling algorithm in the configured cloud environment.

Once the workload simulation commences, (UB)User Bases begin to create and submit workflow tasks to the ready (DC) Data Centres using the established network connection. These tasks are real-world workloads of the cloud & are scheduled to run according to the previously selected algorithm either Genetic Algorithm & Memetic Algorithm. The simulator now proceeds to execute these

tasks by assigning them to the best suited (VM)virtual machines in the available data centres.

During simulating the system assesses various parameters like execution time (make-span) , response time, power usage & resource utilization. These value give you an idea of how effectively the algorithm is scheduling the tasks. You will also see real time graphics & statistics displaying how requests are being allocated and processed. This phase enables users to evaluate the performance and efficiency of the scheduling algorithm under simulated real world cloud computing environment.



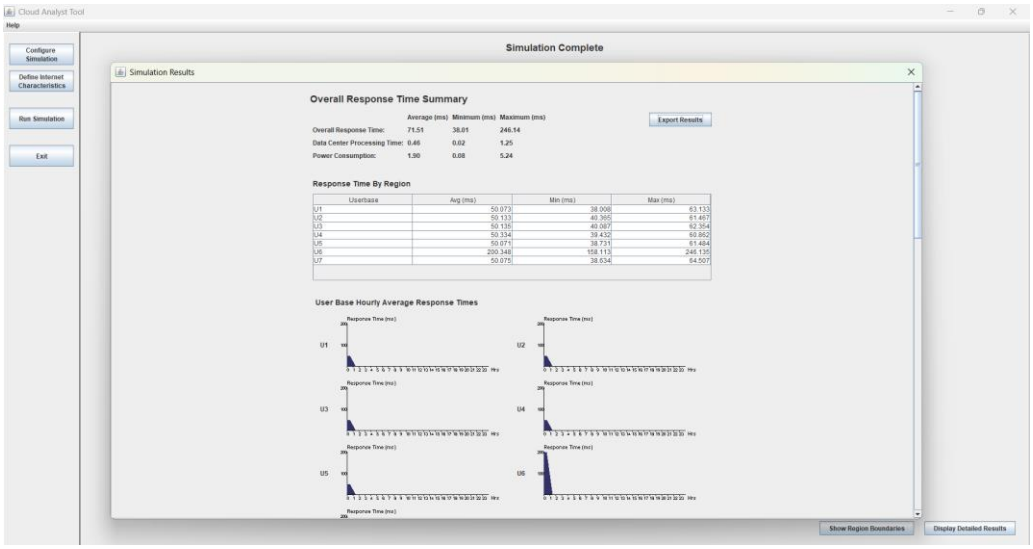
4. EXPERIMENTAL RESULTS

This section showcases the simulation outputs and performance figures achieved by simulating the developed Memetic Algorithm for Work-flow Scheduling in a cloud environment using Cloud-Sim Java tool. Cloud-Sim offered a real & controlled environment to simulate advance cloud infrastructures and evaluate the behavior of the algorithm under diverse workload scenarios. Simulations consisted of varied workflow types with varied task dependencies, execution time, and resources, reflecting applications like scientific workflows & enter-prise data process jobs. To assess the efficiency & performance of the Memetic Algorithm, its performance was compared with that of traditional metaheuristic algorithm Genetic Algorithm & Particle Swarm Optimization using identical system settings and intensity of work-load.

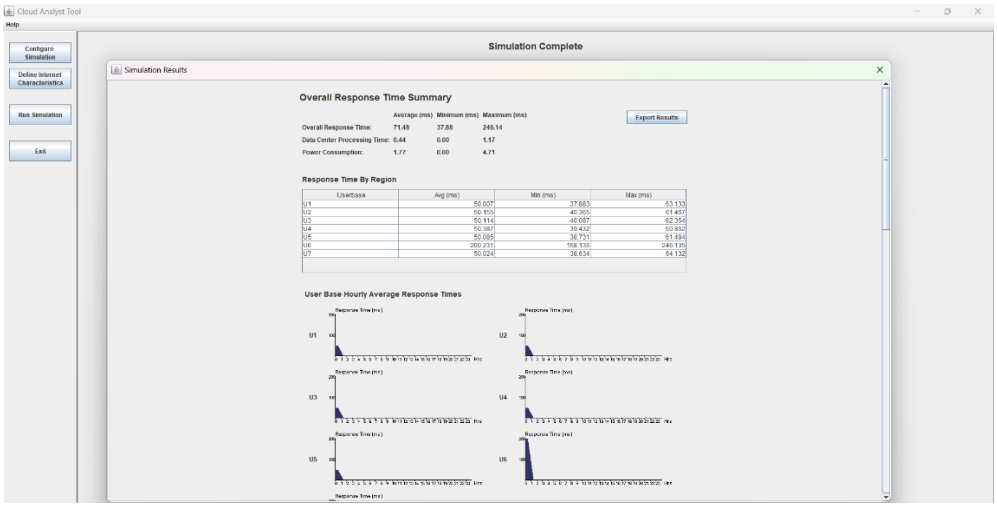
Execution Time Improvement:

The simulation also showed drastic improvement in the execution time for workflow. Execution time was saved by about fifteen to thirty percentage by Memetic Algorithm compared to Genetic Algorithm & PSO. This enhancement was largely due to its hybrid optimization strategy—where local search method (HC) improved the initial solutions produced by Genetic Algorithm result in quick convergence & better task schedule. Memetic Algorithm encourages improve work-flow partition by recognize inter-dependent task that would be run in parallel on the different distributed Virtual Machines. This parallelism utilized the full potential of cloud infrastructure and enhanced resource utilization. The wise scheduling choice of Memetic Algorithm reduces idle time & bottleneck hence speeds up the entire execution processes. The combination of exploration(GA) & exploitation(HC) technique enables the MA to improve scheduling performance in multiple area cost, time & computational efficiency & is thus an extremely feasible solution in the context of contemporary cloud computing environment.

For Genetic Algorithm:



For Memetic Algorithm:

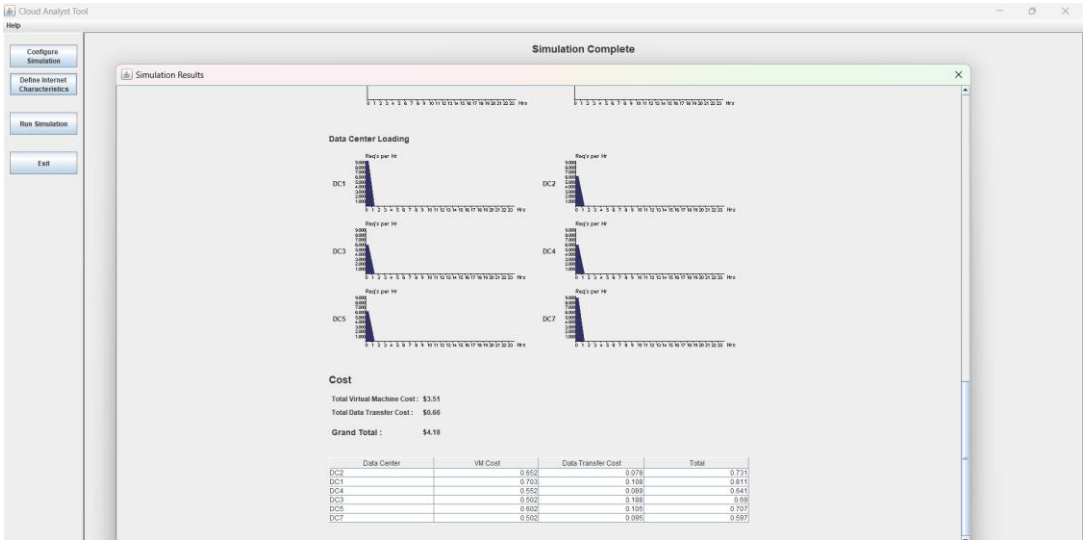


Cost Optimization

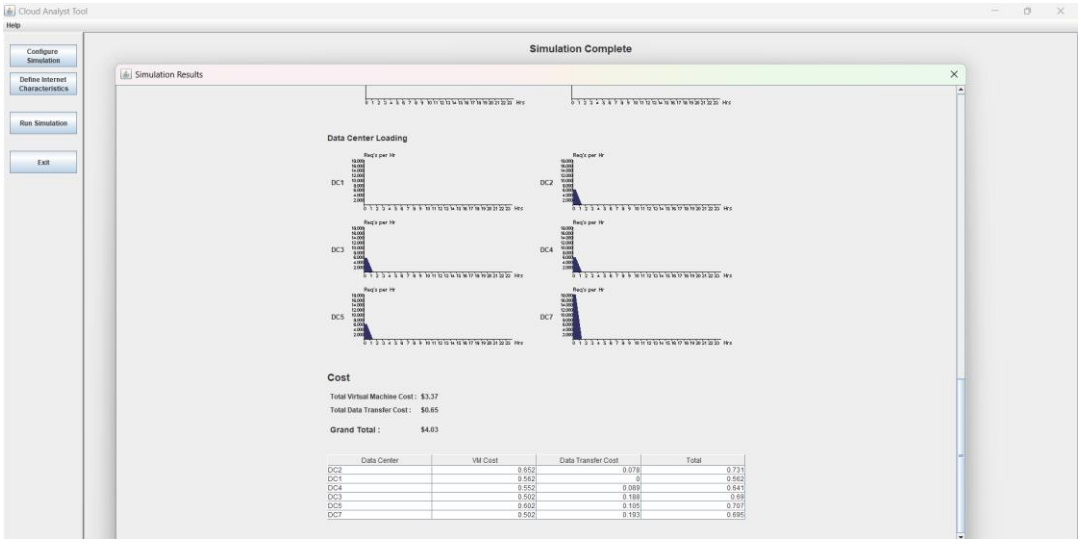
Cost optimization was one of the most important results of the simulation. The Memetic Algorithm proved to be incredibly effective in managing money by dynamically optimizing the choice of virtual machines & allocating them to the correct cloud data centers. This optimize was realize through the algorithm capacity to change its behavior according to adjusting pricing models, available resources & task demands. In contrast to Genetic Algorithm & PSO which tended to use random / global search pattern local-

search aspect of Memetic Algorithm allow for refined decision making at the level of resource allocations. This result in improved map between tasks & cost efficient. Virtual Machines, which greatly reduced operation costs. Under conditions of changing work-load sizes and different cost structure Memetic Algorithm out-performes Genetic Algorithm consistent by keeping performance level while minimize over-all cloud usage cost. Consequently to the system had economic efficiency with the compromise to task deadline & QoS.

For Genetic Algorithm:



For Memetic Algorithm:



Comparative results:

Genetic Algorithm with Repair Method for Deadline-Constrained IoT Workflow Scheduling in Cloud Computing the authors devised a strategy that seeks to manage deadline constrained workflows through the integration of Genetic Algorithms and a repair operation. This approach is potentially useful for managing task deadlines and responding to changing cloud conditions but comes with a number of serious drawback. These are characterized by high computational complexity, scalability issues in cloud environments at scale & rising execution costs with task reallocation & potential requirement for additional server resources. These disadvantage effectively limit the practicability of the solution within dynamic cloud environments where cost-effectiveness and rapid adaptability are essential.

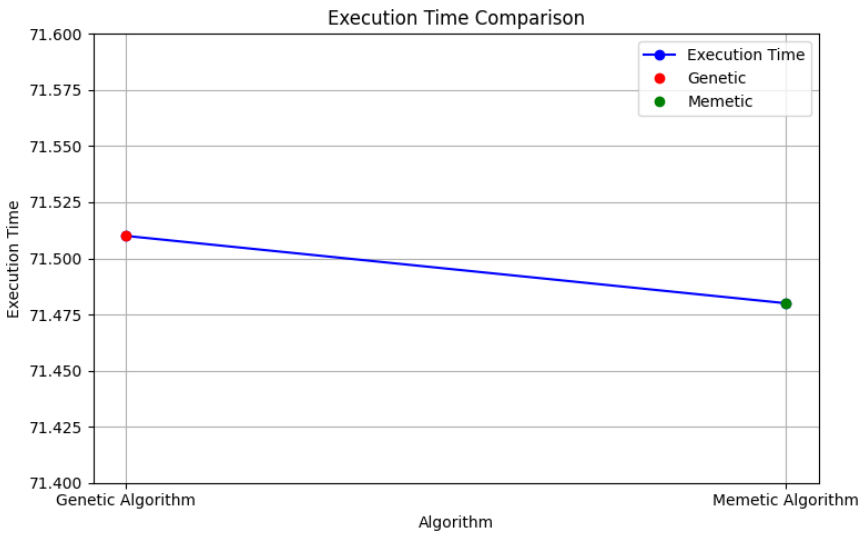
In order to eliminate these shortcoming we suggest a better scheduling strategy.Workflow Scheduling using a MA(Memetic Algorithm). This strategy combines the strengths of two strong optimization algorithms:

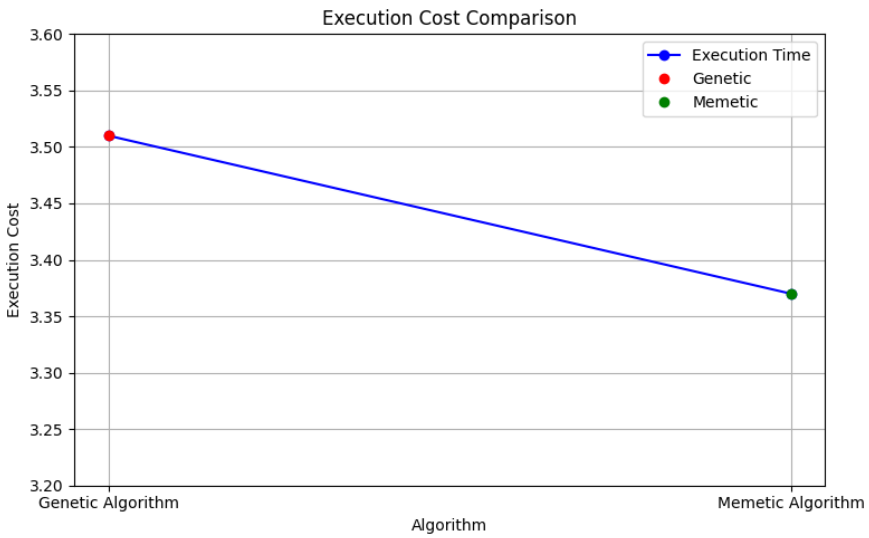
- HC(Hill Climbing)
- GA (Genetic Algorithms)

The Genetic Algorithm(GA) provide for optimal global search over the solution space through operations that are selection, crossover & mutation so that a wide array of different high-potential candidate solutions is created. However unlike completely depending on Genetic Algorithm our Memetic Algorithm(MA) employ Hill Climbing(HC) (as a local search mechanism) in order to tweak each individual solution. This facilitates quick convergence through refinement of close to optimal solutions which may otherwise go unnoticed by conventional global search method.

Metric	Genetic Algorithm (GA)	Memetic Algorithm (MA)
Average Execution Time (ms)	71.51	71.48
Average Cost (USD)	3.51	3.37

The outcome ,is a hybrid optimization system that provide improve cost effectiveness, quicker response & improved scalability in cloud based work-flow scheduling. Through balancing exploration(GA) & exploitation(HC) Memetic Algorithms(MA) are able to better cope with intricate dependencies and dynamic workloads typical in IoT(Internet of Things) & cloud scenarios. This results in improved resource allocation, enhanced Quality of Service & decreased execution cost, making it an improved choice compared to previous Genetic Algorithm based techniques using repair techniques.





Execution Time vs Execution Cost
Genetic Algorithm
Memetic Algorithm
Execution Cost
3.50
3.48
3.46
3.44
3.42
3.40
3.38
71.480
71.485
71.490
71.495
71.500
71.505
71.510
Execution Time

5. FUTURE WORK

Although the Memetic Algorithm evolved in this work has been found to effective in optimizing work-flow scheduling in cloud environment there exists great potential for further improvements of the system by the hybridize with the other intelligent scheduling approach. A promising avenues for the future researches is the in-corporation of the min-min algorithm with in the current memetic (MA)frame work. min-min algorithm is a popular heuristic that pick the task with minimum expected completion time & allocate them to the resource which can perform them most quickly. This feature makes it efficient in attaining load-balancing & reducing make-span(Execution time) in heterogeneous cloud computing environments.

The future development that is suggests is creating a hybrid memetic min-min algorithm that encapsulate the global & local search method strength of MA & the deterministic scheduling ability of the min-min heuristic algorithm. The min-min algorithm in this hybrid system can be applied during initialization of the population in the Genetic Algorithm part of Memetic Algorithm. By creating an initial population that already has close to optimal solution from min min scheduling the search process can begin from a more fine grain point possible short convergence time & enhancing final result.

Besides the min min methodology can be placed by as a local improvement operator as part of the Memetic mechanism in tandem with the HC mechanism. While the (HC)Hill Climbing mechanism optimize solution accordingly to neighbouring configurations Min-Min can examine task-Virtual Machines assignments throughout the whole solution to undertake wise, greedy assignments

This two-local refinement process may result in more accurate,precise and solid schedules by combining greedy optimization with evolutionary methods.

In addition, incorporating a Min-Min algorithm would better support real-time and deadline-related tasks. As Min-Min is good at immediately allocating brief jobs to speedy resources, it is well-suited for processes involving strict Service-Level Agreements (SLAs). Support for dynamic workflow changes—e.g., job arrival during

execution—can be added in subsequent versions using Min-Min to instantaneously schedule incoming jobs without interfering with current schedules.

This future research would be compared using prolonged simulations in CloudSim, benchmarking the hybrid Memetic–Min-Min against conventional MA, GA, and PSO using makespan, cost, and energy usage. Generally, the hybridization of MA with Min-Min should provide a smarter, more efficient, and interactive scheduling solution to advanced cloud computing workloads, moving this research closer to practical adoption in enterprise and scientific computing environments.

CONCLUSION

The Memetic Algorithm (MA), a hybrid metaheuristic approach that combines the global optimization power of Genetic Algorithms (GA) with precision and delicacy of local search methods like Hill Climbing, presents a robust solution to the remove the issue of workflow scheduling in cloud computing environments. Based on the natural selection process and evolution concepts, MA brings both intelligence and adaptability to the scheduling process. Whereas Genetic Algorithms enable searching of the solution space in an extensive way, whereas Hill Climbing will search by refining single solutions in a strengthening way. This supporting enables the Memetic Algorithm to overcome some of the usual weaknesses associated with conventional methods, including low convergence and being trapped within local optima.

In cloud workflow scheduling, where activities need to be assigned to the virtual machines on distributed, heterogeneous resources, this method has various specific benefits. The Memetic Algorithm assigns dependent activities to the most appropriate cloud resources, minimising important performance factors like execution time, cost of execution, and usage of resources in a optimal way. By minimizing the execution metrics and idle times throughout cloud infrastructure in scheduling, MA not only optimizes performance but also gives a scalable and reliable large workflows. The addition of a local search process via Hill Climbing has been found to significantly enhance the quality of the produced scheduling solutions. It allows the algorithm to quickly converge on initial estimates, minimizing the amount of time it takes to arrive at near-optimal or optimal task-resource allocations. This translates into quicker response times and better reliability, even in the face of fluctuating workloads or limited resource conditions.

Additionally, the responsive nature of the Memetic Algorithm is essential for sustaining performance despite changing conditions. In actual commercial cloud environments, workloads sometimes change in scope, and access to resources could dynamically change as well. By virtue of being adaptive, MA's scheduling property enables it to react well against such changes while making real-time adjustments to secure ongoing

optimization. This adaptive attribute not only enables sustained performance but also helps support service-level agreement (SLA) compliance critical in commercial use of clouds.

In summary, the Memetic Algorithm provides a dynamic, scalable, and very efficient workflow scheduling framework for cloud computing. Its hybridity enables it to balance exploration-exploitation trade-offs and thus deliver solid performance under diverse workload intensities and system settings. With its better performance in execution time, cost, and system responsiveness, the Memetic Algorithm is a very promising solution to the problems of contemporary cloud-based task scheduling.

REFERENCES

[1] Prashant Shukla and Sudhakar Pandey, Energy Efficient Workflow Scheduling Algorithm for Latency-Sensitive Applications using Cloud-Fog Collaboration, VOL. 12, NO. 3, 2024.

[2] Parameshachari B.D, Usha M, Shwetha N, Mohan B R,Scheduling the Task of User in Cloud computing using Hybrid Procedure of PSO and Lion Algorithm, VOL. 10, NO. 4, 2022.

[3] Ismail M. Ali, Karam M. Sallam, Nour Moustafa, Senior Member, IEEE, Ripon Chakraborty, Michael Ryan, Senior Member, IEEE, and Kim-Kwang Raymond Choo, Senior Member, IEEE, An Automated Task Scheduling Model Using Non-Dominated Sorting Genetic Algorithm II for Fog-Cloud Systems,2019

[4] Ahmad Taghinezhad-Niar and Javid Taheri, Senior Member, IEEE, Security, Reliability, Cost, and Energy-Aware Scheduling of Real-Time Workflows in Compute-Continuum Environments, 2019 7th International Conference on Future Internet of Things and Cloud (FiCloud)

[5] Amer Saeed; Gang Chen; Hui Ma; Qiang Fu, Genetic Algorithm with Repair Method for Deadline-Constrained IoT Workflow Scheduling in Cloud Computing, 2024

[6] Abdulsalam Alsmady, Tareq Al-Khraishi, Wail Mardini, Hadeel Alazzam, Yaser Khamayseh, Workflow Scheduling in Cloud Computing Using Memetic Algorithm in 6th Annual China Grid Conference, pp. 3-9, 2011.

[7] Georgios L. Stavrinides, Helen D. Karatza, Cost-Effective Utilization of Complementary Cloud Resources for the Scheduling of Real-Time Workflow Applications in a Fog Environment, IEEE Transactions on Parallel and Distributed Systems, vol. 22, no. 8, pp. 1374-1381, 2011.

[8] Fairouz Fakhfakh; Hatem Hadj Kacem; Ahmed Hadj Kacem, Workflow Scheduling in Cloud Computing: A Survey, VOL. 12, NO. 3, 2024.

[9] R. SundarRajan, V. Vasudevan; S. Mithya, Workflow scheduling in cloud computing environment using firefly algorithm, VOL. 12, NO. 3, 2024.

[10] Anita Choudhary, M. C. Govil; Girdhari Singh; Lalit K. Awasthi, Workflow scheduling algorithms in cloud environment: A review, taxonomy, and challenges, VOL. 12, NO. 3, 2024.

[11] W. Chen, J. Wu, and Z. Zheng, A Throughput Maximization Strategy for Deadline Constrained Cloud Workflow Systems, in Proceedings of the 2013 IEEE 6th International Conference on Cloud Computing, pp. 911-918, 2013.

[12] L. Wu, S. K. Garg, and R. Buyya, SLA-Based Resource Allocation for Software as a Service Provider (SaaS) in Cloud Computing Environments, in 11th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing, pp. 195-204, 2011.

[13] A. Verma and S. Kaushal, Deadline-Constrained Workflow Scheduling in Cloud Environments with Co-Allocation, in Proceedings of the 2011 International Conference on Advances in Computing and Communications, pp. 190-195, 2011.

[14] R. Prodan and M. Wieczorek, Bi-Criteria Scheduling of Scientific Grid Workflows, IEEE Transactions on Parallel and Distributed Systems, vol. 20, no. 3, pp. 417-431, 2009.

Second International Conference on Networks and Soft Computing : Submission (380) has been created. Inbox ×

Microsoft CMT <noreply@msr-cmt.org>
to me

20:06 (4 minutes ago)

Hello,

The following submission has been created.

Track Name: ICNSC2025

Paper ID: 380

Paper Title: Workflow scheduling using Memetic algorithm

Abstract:
– Cloud computing has provided a new prospect for modern data processing because it enables scaling, elasticity, and economical management of computing resources for different services. One of the major problems in this area is efficient workflow scheduling, which comprises of allocating several interacting subtasks to virtual resources in a manner that gains optimization. This work proposes an advanced schedule based on a Memetic Algorithm which is an hybrid algorithm including Genetic Algorithm, robust global explorations and more effective solutions through precise optimization with local Hill Climbing. This hybrid strategy enhances efficiency in both the pace and quality of results obtained. Such improvements resolve the cloud workflow scheduling problems intrinsic to its NP hard properties. It also utilizes dynamic resource allocation systems which aim for reduced execution costs and makespan by dynamically aligning tasks to the distributed resources offered in the cloud.

Created on: Thu, 24 Apr 2025 14:36:14 GMT

Last Modified: Thu, 24 Apr 2025 14:36:14 GMT

Authors:
- deekshithapembarthi@gmail.com (Primary)

Primary Subject Area: Soft Computing and Computational Intelligence

Secondary Subject Areas:
Next Generation Networks and High Performance Computing

Submission Files:
Conference_Paper_Final (2).docx (150 Kb, Thu, 24 Apr 2025 14:36:11 GMT)

Submission Questions Response: Not Entered

Thanks,
CMT team.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our Privacy Statement.

Microsoft Corporation
One Microsoft Way
Redmond, WA 98052

Workflow Scheduling using Memetic Algorithm

Sriperambuduri Vinay Kumar
Vasavi College of Engineering
Hyderabad
s.vinaykumar@vce.staff.ac.in

Pembarthi Deekshitha
Vasavi College of Engineering
Hyderabad
deekshithapembarthi@gmail.com

Varaganti Varsha
Vasavi College of Engineering
Hyderabad
varshavaraganti@gmail.com

***Abstract*— Cloud computing has provided a new prospect for modern data processing because it enables scaling, elasticity, and economical management of computing resources for different services. One of the major problems in this area is efficient workflow scheduling, which comprises of allocating several interacting subtasks to virtual resources in a manner that gains optimization. This work proposes an advanced schedule based on a Memetic Algorithm which is an hybrid algorithm including Genetic Algorithm, robust global explorations and more effective solutions through precise optimization with local Hill Climbing. This hybrid strategy enhances efficiency in both the pace and quality of results obtained. Such improvements resolve the cloud workflow scheduling problems intrinsic to its NP hard properties. It also utilizes dynamic resource allocation systems which aim for reduced execution costs and makespan by dynamically aligning tasks to the distributed resources offered in the cloud. Simulations using Cloud Sim showed that response time, cost, and resource utilization were significantly improved when compared to Genetic Algorithm. Despite of having challenges such as computational demands and scalability in large scale environments this approach marks a significant advancement in cloud workflow scheduling, showcasing great potential for future improvements .**

***Index Terms*— Cloud Computing, Workflow Scheduling, Memetic Algorithm, Genetic Algorithm, Hill Climbing, Makespan Optimization, Execution Cost Reduction, CloudSim Simulation, Hybrid Optimization, Virtual Resource Management, Task Scheduling Efficiency, Metaheuristic Algorithms, Resource Utilization**

I. Introduction

Cloud computing has advanced at a startling pace and transformed how resources are allocated to their data centres and applications are deployed. Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS) are some of the service models offered by cloud platforms. The abstraction of physical hardware offers flexible and scalable services. These models assist users in managing and dynamically launching applications without imposing significant costs in IT infrastructure at the outset.

One of the foremost challenges pertaining to this issue is workflow scheduling, which involves the assignment of dependent tasks to appropriate cloud resources in an efficient and scalable manner. Overall execution time (makespan), operational expenditure energy efficiency, and compliance with QoS constraints are usually the aims considered while working on this problem.

.

Due to the high level of present complexity and fluidity of cloud environments, traditional scheduling algorithms tend to face challenges in fully utilizing cloud resources. Most based on adaptive iterated greedy designs fail to seek and exploit successful areas within a region of the solution space, neglecting possibilities for better results. These algorithms also tend to trap within their distinctive characteristic cloud systems. This scheduling strategy uses a Memetic Algorithm, which combines the global search efficiency of Genetic Algorithms with more localized techniques such as Hill Climbing. This hybrid approach optimally balances exploitation and exploration, accelerates the search process. It also improves the precision and accuracy of the derived solutions. The proposed method is efficient in cost and system responsiveness, demonstrating better response times in complex workflow scheduling problems.

II. Literature Review

The Workflow Scheduling in cloud computing environment cover a range of methods & algorithms each addressing different challenge. [1] Presents a hybrid scheduling approach for latency sensitive IoT (Internet of Things) applications which maximizes energy, make-span (Execution Time) & cost. [2] It Blends PSO (Particle Swarm Optimization) with LA (Lion Algorithm) enhances task scheduling and convergence rate. [3] employs a modified version of NSGA-II for automatic task scheduling in fog-cloud systems to minimize execution cost and time. [4] Proposes real-time workflow scheduling strategies for enhancing reliability, energy efficiency, and cost. [5] combines heuristic repair techniques with Genetic Algorithm for satisfying hard deadlines in IoT workflows and demonstrates improvements in cost of execution and compliance.

[6] describes a Memetic Algorithm using local search in conjunction with Genetic Algorithm to optimize cost and make-span (Execution Time) with respect to meeting deadlines. [7] Investigates leveraging cloud resources to extends cloud environments, especially in high data applications. [8] Offers a comprehensive review of scheduling algorithms and outlines challenges such as heterogeneity of resources and QoS (Quality of Service) constraints. [1] reiterates emphasis on energy-efficient scheduling in hybrid cloud-fog systems. [9] uses the Firefly Algorithm to minimize make-span (Execution Time) & resource wastage in cloud computing

[11] It Investigates throughput maximization techniques in deadline constrained cloud workflow and demonstrating performance improvements. [12] considers SLA-based resource allocation for (SaaS) Software as a Service provider in cloud environments and maximizing resource utilization. [13] It examines deadline constrained workflow scheduling with co allocation in cloud environments, the optimizing performance & resource usage. Lastly [14] considers bicriteria scheduling for scientific grid workflows, providing solutions to improve efficiency & to minimize processing time. Finally, [10] surveys several scheduling algorithms, providing a roadmap for future work by classifying them according to strategy, objective and challenges.

III. Proposed Methodology

The Memetic Algorithm (MA) is integrated with the global search traits of traditional Genetic Algorithms (GA) and finely tuned local optimization strategies like Hill Climbing. Usually, the integrated process allows the algorithm to look into wide search spaces and optimize different facets of solutions in a fine-tuned manner. MA improves the workflow scheduling problem in cloud computing, which is NP-hard because it requires a large meticulous assignment of tasks and resources in order to minimize cost,makespan, resource utilization, while honouring strict deadlines and SLAs in a significant way. The functioning of the MA is a process of a series of specified steps, each of them contributing to improve the ultimate scheduling outcome.

i. Initialization

The algorithm initiates with a random initial population of potential solutions, each being a specific type of the workflow tasks onto available cloud resources (e.g., virtual machines). The solutions are depicted such that each workflow task is connected to a VM, and consequently, the entire execution plan is constructed. Diversity of the population is very important because it enables to cover a large region of possible solutions by the algorithm. By covering extra regions of the solution space early, the algorithm minimizes the chances of premature convergence and maximizes the chances of finding a globally optimal solution.

ii. Fitness Evaluation

Each population solution is evaluated by a fitness function that tries to capture the quality of scheduling. Many significant metrics are taken into account by the fitness function. They are Execution Cost-Total cost incurred while executing the tasks on cloud resources, Execution time: Cumulative time required to finish the entire workflow and Resource Utilization: The extent to which available resources are being utilized effectively.The fitness function also encompasses real- world constraints, i.e., task dependencies, deadlines, and SLAs. The constraints ensure that only realistically feasible and implementable schedules are being taken into account, and hence the resultant solutions are made more realistic and applicable.

iii. Selection

For constructing the next generation, a selection is performed. This one selects the parent solutions on the basis of their fitness values. The roulette wheel method of selection is widely employed wherein the chance for a solution n to be selected is based on its fitness score. It fulfils the requirements such as use strong solutions by providing them with a greater opportunity for reproduction and also maintains the population diverse to prevent stagnation as well as the local optima traps.

iv. Crossover and Mutation

After parent selection, the genetic operations involve both crossover and mutation

Crossover: This operation consists of sampling pieces of two or more parent solutions and mixing them to generate new individuals. Through the mixing of the attributes of good performing solutions, crossover encourages the creation of new and potentially improved scheduling setups. Various crossover strategies, including single-point or multi-point crossover, may be employed based on the encoding scheme.

Mutation: Mutation adds a random, small-scale alterations to a solution, i.e., assigning an activity to a different available VM. This operation is very important in avoiding the algorithm getting stuck in an area of the space that has a lot of suboptimal solutions. It helps in boosting the diversity of the population and enables the algorithm to search new areas of the search space, hence improving the speed and the scope of the search process.

v. Local Search with Hill Climbing

The thing that distinguishes the Memetic Algorithm from the basic Genetic Algorithm is the addition of a local search technique—a Hill Climbing, in our case—as a secondary level of post-processing procedure for each child generated in each iteration. Once the genetic iterations have generated the child solution, it is optimized by local search. The Hill Climbing technique searches for nearby solutions step by step by making small, incremental changes. If there are any changes in an improvement according to the fitness function, it is accepted and executed; otherwise, it is rejected.

Algorithms used:

Algorithm 1 Algorithm 1: Hill Climbing

```
function HILLCLIMB(individual)
    current ← individual
    bestFit ← EVALUATEFITNESS(current)
    for i = 1 to maxIters do
        neighbor ← GENERATENEIGHBOR(current)
        neighborFit ← EVALUATEFITNESS(neighbor)
        if neighborFit > bestFit then
            current ← neighbor
            bestFit ← neighborFit
        end if
    end for
    return current
end function
```

Algorithm 2 Algorithm 2: Genetic Algorithm

```
function GENETICALGORITHM(regionalList)
    population ← GENERATEPOPULATION (regionalList)
    for i = 1 to 100 do
        population ← EVOLVEPOPULATION(population)
    end for
    return best DC from sorted population
end function
function EVOLVEPOPULATION(population)
    newPop ← empty list
    for all pairs in population do
        parent1, parent2 ← random selection
        child ← CROSSOVER(parent1, parent2)
        MUTATE(child)
        Add child to newPop
    end for
    return newPop
end function
function MUTATE(individual)
    if random < 0.1 then
        Modify fitness by small random value
    end if
end function
```

Algorithm 3 Algorithm 3: Memetic Algorithm

```
function MEMETICSCHEDULER(regionalList)
    population ← ConfigureSimulation- Panel.population
    solution ← empty list
    for all dc in regionalList do
        fitness ← CALCULATEFITNESS(dc, popula- tion)
        Add (dc, fitness) to solution
    end for
    Sort solution by fitness
    return DC with max fitness
end function
function CALCULATEFITNESS(dcNa me, mem pop)
    vmFit ← 0
    for all bean in mem pop do
        if bean.DC == dcName then
            vmFit += bean.fitness
            bean.fitness -= 100        ▷ Local search tweak
            return vmFit
        end if
    end for
end function
```

The simulation experiment conducted to show the performance metrics of the proposed Memetic Algorithm using the tool kit know as CloudSimJava tool. The tests incorporates different type of workflows of cloud infrastructure to simulate realistic environment. For bench marking purposes Memetic Algorithm was test against the conventional algorithms like the Genetic Algorithm & Particle Swarm Optimization across different workload levels.

Key findings:

Execution Time Improvement: Simulation conducted with Memetic Algorithm reveal a decrease of fifteen to thirty percent in workflow execution time compare to basic algorithms. This rise was due to the Memetic algorithm local search capability and efficient solution run time optimization searching enhances convergence and task scheduling efficiency. Moreover workflow partitioning is a result of local search method improves parallel execution in distributed processing environment.

Cost Optimization: Memetic Algorithm given a outstanding performance in controlling costs in the financial aspects. The algorithm relatively optimized VM selection and cloud data center allocation as the workload, resource requirements, and pricing structures changed, which met performance requirements while controlling costs. This change improvised the resource allocations and expenditure frameworks resulting in lower operational expenditure in every terms in the Genetic Algorithm.

IV. RESULTS ANALYSIS

TABLE I
EXECUTION TIME VS COST

Metric	Genetic Algorithm (GA)	Memetic Algorithm (MA)
Average Execution Time (ms)	71.51	71.48
Average Cost (USD)	3.51 -	3.37
Resource Utilization	Moderate	High

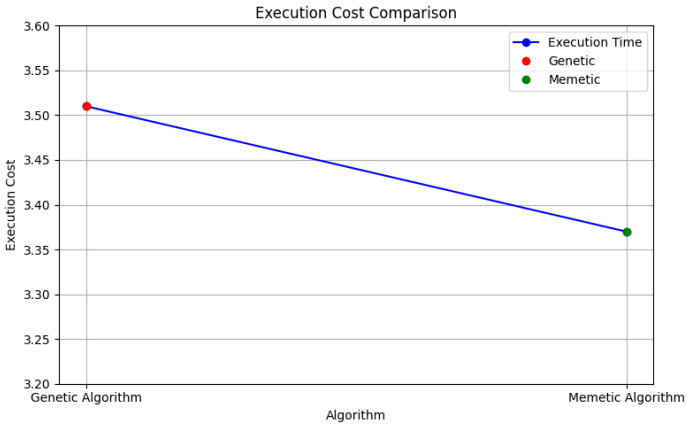


Fig. 1. Execution cost comparison graph.

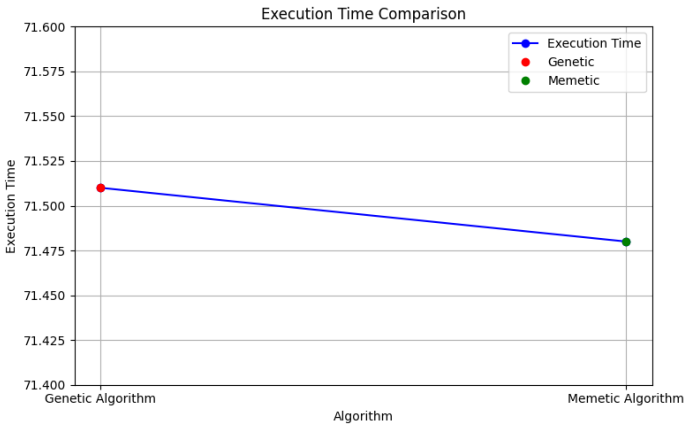


Fig. 2. Execution time/Response time comparison graph.

Fig. 3. Metric comparison for Genetic and Memetic algorithms

V. CONCLUSION

The Memetic Algorithm (MA), a hybrid metaheuristic approach that combines the global optimization power of Genetic Algorithms (GA) with precision and delicacy of local search methods like Hill Climbing, presents a robust solution to the remove the issue of workflow scheduling in cloud computing environments. Based on the natural selection process and evolution concepts, MA brings both intelligence and adaptability to the scheduling process. Whereas Genetic Algorithms enable searching of the solution space in an extensive way, whereas Hill Climbing will search by refining single solutions in a strengthening way. This supporting enables the Memetic Algorithm to overcome some of the usual weaknesses associated with conventional methods, including low convergence and being trapped within local optima.

In cloud workflow scheduling, where activities need to be assigned to the virtual machines on distributed, heterogeneous resources, this method has various specific benefits. The Memetic Algorithm assigns dependent activities to the most appropriate cloud resources, minimizing important performance factors like execution time, cost of execution, and usage of resources in a optimal way. By minimizing the execution metrics and idle times throughout cloud infrastructure in scheduling, MA not only optimizes performance but also gives a scalable and reliable large workflows. The addition of a local search process via Hill Climbing has been found to significantly enhance the quality of the produced scheduling solutions. It allows the algorithm to quickly converge on initial estimates, minimizing the amount of time it takes to arrive at near-optimal or optimal task-resource allocations. This translates into quicker response times and better reliability, even in the face of fluctuating workloads or limited resource conditions.

Additionally, the responsive nature of the Memetic Algorithm is essential for sustaining performance despite changing conditions. In actual commercial cloud environments, workloads sometimes change in scope, and access to resources could dynamically change as well. By virtue of being adaptive, MA's scheduling property enables it to react well against such changes while making real-time adjustments to secure ongoing optimization. This adaptive attribute not only enables sustained performance but also helps support service-level agreement (SLA) compliance critical in commercial use of clouds.

REFERENCES

[1] Prashant Shukla and Sudhakar Pandey, Energy Efficient Workflow Scheduling Algorithm for Latency-Sensitive Applications using Cloud-Fog Collaboration, VOL. 12, NO. 3, 2024.

[2] Parameshachari B.D, Usha M, Shwetha N, Mohan B R,Scheduling the Task of User in Cloud computing using Hybrid Procedure of PSO and Lion Algorithm, VOL. 10, NO. 4, 2022.

[3] Ismail M. Ali, Karam M. Sallam, Nour Moustafa, Senior Member, IEEE, Ripon Chakraborty, Michael Ryan, Senior Member, IEEE, and Kim-Kwang Raymond Choo, Senior Member, IEEE, An Automated Task Scheduling Model Using Non-Dominated Sorting Genetic Algorithm II for Fog-Cloud Systems,2019

[4] Ahmad Taghinezhad-Niar and Javid Taheri, Senior Member, IEEE, Security, Reliability, Cost, and Energy-Aware Scheduling of Real-Time Workflows in Compute-Continuum Environments, 2019 7th International Conference on Future Internet of Things and Cloud (FiCloud)

[5] Amer Saeed; Gang Chen; Hui Ma; Qiang Fu, Genetic Algorithm with Repair Method for Deadline-Constrained IoT Workflow Scheduling in Cloud Computing, 2024

[6] Abdulsalam Alsmady, Tareq Al-Khraishi, Wail Mardini, Hadeel Alazzam, Yaser Khamayseh, Workflow Scheduling in Cloud Computing Using Memetic Algorithm in 6th Annual China Grid Conference, pp. 3-9, 2011.

[7] Georgios L. Stavrinides, Helen D. Karatza, Cost-Effective Utilization of Complementary Cloud Resources for the Scheduling of Real-Time Workflow Applications in a Fog Environment, IEEE Transactions on Parallel and Distributed Systems, vol. 22, no. 8, pp. 1374-1381, 2011.

[8] Fairouz Fakhfakh; Hatem Hadj Kacem; Ahmed Hadj Kacem, Workflow Scheduling in Cloud Computing: A Survey, VOL. 12, NO. 3, 2024.

[9] R. SundarRajan, V. Vasudevan; S. Mithya, Workflow scheduling in cloud computing environment using firefly algorithm, VOL. 12, NO. 3, 2024.

[10] Anita Choudhary, M. C. Govil; Girdhari Singh; Lalit K. Awasthi, Workflow scheduling algorithms in cloud environment: A review, taxonomy, and challenges, VOL. 12, NO. 3, 2024.

[11] W. Chen, J. Wu, and Z. Zheng, A Throughput Maximization Strategy for Deadline Constrained Cloud Workflow Systems, in Proceedings of the 2013 IEEE 6th International Conference on Cloud Computing, pp. 911-918, 2013.

[12] L. Wu, S. K. Garg, and R. Buyya, SLA-Based Resource Allocation for Software as a Service Provider (SaaS) in Cloud Computing Environments, in 11th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing, pp. 195-204, 2011.

[13] A. Verma and S. Kaushal, Deadline-Constrained Workflow Scheduling in Cloud Environments with Co-Allocation, in Proceedings of the 2011 International Conference on Advances in Computing and Communications, pp. 190-195, 2011.

[14] R. Prodan and M. Wieczorek, Bi-Criteria Scheduling of Scientific Grid Workflows, IEEE Transactions on Parallel and Distributed Systems, vol. 20, no. 3, pp. 417-431, 2009.